

PROJETO DE FÃ · PORTFÓLIO

CATARATAS DO IGUAÇU

Documentação de sistema: especificação de requisitos, arquitetura, segurança, testes e operação

Uma homenagem de fã. Este sistema não representa o site real das Cataratas do Iguaçu nem o Parque Nacional do Iguaçu. É exclusivamente um projeto de portfólio, com dados e vendas fictícios. · Versão 2.2 · 2 de novembro de 2026

Controle do documento

Título	Cataratas do Iguaçu · Documentação de Sistema (SRS, arquitetura, testes e operação)
Código	CPI-DOC-001
Versão	2.2
Data	2 de novembro de 2026
Situação	Versão final do portfólio
Autoria	Fã do parque, autor do projeto
Produto descrito	Sistema web Cataratas do Iguaçu, v1.0 (front-end Angular, API Node.js, Docker)
Normas de referência	ISO/IEC/IEEE 29148:2018, ISO/IEC/IEEE 42010:2022, ISO/IEC 25010, modelo C4
Classificação	Público. Projeto de portfólio, sem vínculo com o Parque Nacional do Iguaçu.
Fontes	Pasta docs/livro do repositório (gerar.py regenera o PDF)

Histórico de revisões

Versão	Data	Descrição
0.1	2026-11-02	Migração do site HTML para Angular e criação do back-end Node.js.
1.0	2026-11-02	Primeiro livro: visão geral, requisitos, arquitetura e operação.
2.0	2026-11-02	Reestruturação conforme ISO/IEC/IEEE 29148 e 42010: partes, critérios de aceitação, casos de uso, diagramas C4 e UML, ADRs, modelo de ameaças, plano de testes e matriz de rastreabilidade.
2.1	2026-11-02	Testes ponta a ponta versionados (Playwright), fluxo de CI e correção de acessibilidade (WCAG 2.5.3); rastreabilidade atualizada.
2.2	2026-11-02	Refatoração em camadas (back-end e front-end), estilos consolidados, ESLint nos três projetos, imagens otimizadas e 12 testes de API, CLI e unidade; ADR-11, ADR-12 e convenções de código.

Aviso importante

Este documento e o sistema que ele descreve foram criados por um **fã**, como **homenagem** às Cataratas do Iguaçu e ao Parque Nacional do Iguaçu, Patrimônio Mundial Natural da UNESCO. O sistema **não representa um site real** das Cataratas do Iguaçu, não é mantido, aprovado ou endossado pelo parque ou por qualquer órgão responsável e **trata-se apenas de um portfólio**. Ingressos, preços, pagamentos, e-mails e contatos são fictícios ou ilustrativos; nenhuma compra real é realizada. Para informações oficiais, consulte sempre os canais oficiais do parque. As fotografias pertencem ao acervo do projeto educacional original, sem créditos de autoria informados.

Sumário

PARTE I · CONTEXTO E VISÃO DO PRODUTO 4

01 Introdução 5

02 Visão do produto 7

PARTE II · ESPECIFICAÇÃO DE REQUISITOS 10

03 Descrição geral e interfaces 11

04 Requisitos funcionais 19

05 Requisitos não funcionais 22

06 Regras de negócio 24

07 Casos de uso 26

08 Modelo de dados 30

PARTE III · PROJETO E IMPLEMENTAÇÃO 33

09 Arquitetura 34

10 Front-end Angular 40

11 Back-end e API 43

12 Segurança 45

13 E-mail e comprovante 47

14 Administração e CLI 49

PARTE IV · QUALIDADE, OPERAÇÃO E ENCERRAMENTO 50

15 Estratégia e plano de testes 51

16 Matriz de rastreabilidade 54

17 Implantação e operação 55

18 Experiência do usuário e acessibilidade 57

19 Riscos residuais, limitações e evolução 59

A Glossário 61

B Referências 62



PARTE I

Contexto e visão do produto

Por que o sistema existe, para quem e com quais limites.

01 · Introdução
02 · Visão do produto

CAPÍTULO 1

Introdução

1.1 Finalidade do documento

Este livro é a **documentação de sistema** do *Cataratas do Iguaçu*. Ele reúne, em um único documento, a **especificação de requisitos de software** (SRS), a **descrição da arquitetura** (SAD/SDD), a estratégia de testes, a rastreabilidade e o guia de operação. A estrutura segue as recomendações da **ISO/IEC/IEEE 29148:2018** (engenharia de requisitos), da **ISO/IEC/IEEE 42010:2022** (descrição de arquitetura) e do modelo de qualidade da **ISO/IEC 25010**, adaptadas ao porte de um projeto de portfólio.

> **Projeto de fã, apenas portfólio.** O sistema descrito aqui foi criado por um fã, para homenagear as Cataratas do Iguaçu. Ele **não representa o site real** do parque, não tem vínculo com o Parque Nacional do Iguaçu nem com seus órgãos gestores e não realiza vendas reais. Ingressos, preços, pagamentos, e-mails e contatos são fictícios ou ilustrativos. As fotografias do livro provêm do acervo do projeto educacional original, sem créditos de autoria informados no repositório.

1.2 Escopo do produto

Dentro do escopo	Fora do escopo
• Site institucional com informações do parque e atividades de lazer • Cadastro, autenticação e perfis (usuário e superadmin) • Venda fictícia de ingressos com QR Code e comprovante por e-mail • Painel administrativo e CLI de superadmin • Empacotamento e execução com Docker	• Pagamento real e emissão fiscal • Validação de entrada na portaria (leitura do QR Code) • Controle de capacidade e calendário por dia • Redefinição de senha por e-mail • Qualquer integração oficial com o parque

1.3 Público-alvo e como usar o documento

Leitor	Capítulos recomendados
Avaliador / recrutador	1, 2, 3 e 19 (visão geral, telas e limitações)
Analista de requisitos / QA	4 a 8, 15 e 16 (requisitos, regras, casos de uso, testes, rastreabilidade)
Desenvolvedor	9 a 14 (arquitetura, front-end, API, segurança, e-mail, CLI)
Operação / DevOps	17 (implantação e operação) e 12 (segurança)
Designer / UX	3, 10 e 18 (telas, sistema de design e acessibilidade)

1.4 Convenções

- **Identificadores:** RF (requisito funcional), RNF (não funcional), RN (regra de negócio), UC (caso de uso), ADR (decisão arquitetural), TC (caso de teste), R (risco).
- **Prioridade (MoSCoW):** *Must* (essencial), *Should* (importante), *Could* (desejável).
- **Critérios de aceitação:** escritos no formato *Dado / Quando / Então* e verificáveis.
- **Valores monetários:** sempre fictícios, em reais, armazenados em centavos.

1.5 Referências normativas e técnicas

As normas e guias abaixo orientaram a organização do documento e as decisões técnicas. O texto declara *alinhamento* às práticas, e não certificação ou conformidade formal. A lista completa está no Apêndice B.

CAPÍTULO 2

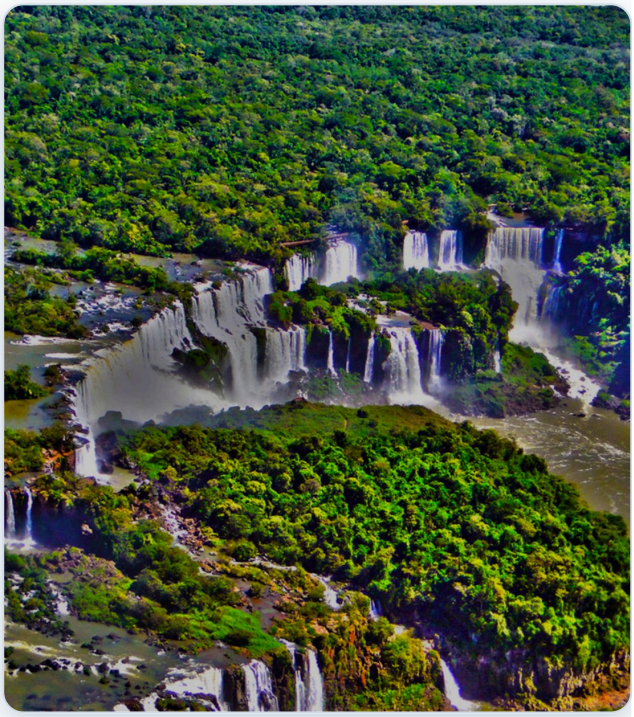
Visão do produto

2.1 Problema e oportunidade

O Parque Nacional do Iguaçu e suas cataratas inspiram um interesse enorme, mas a experiência digital de um visitante envolve várias etapas: entender horários e atrações, escolher ingresso, comprar e levar o comprovante. O projeto propõe uma jornada fluida e bonita para essas etapas e, ao mesmo tempo, serve como **vitrine técnica** de um sistema completo: interface moderna, API segura, painel administrativo, e-mail transacional e operação em contêineres.



Cataratas do Iguaçu vistas do alto.



Quedas e mata atlântica ao redor.

2.2 Partes interessadas

Parte interessada	Interesse	Influência
Visitante / usuário	Informação clara e compra de ingresso simples, em celular ou computador	Alta (define a usabilidade)
Superadmin	Manter ingressos, preços, atividades; acompanhar pedidos e usuários	Alta
Desenvolvedor / mantenedor	Código legível, testes, execução reproduzível, segurança	Alta
Avaliador de portfólio	Evidências de boas práticas de engenharia e de UX	Média

Parte interessada	Interesse	Influência
Parque Nacional do Iguaçu	Não envolvido; o projeto é homenagem e não possui vínculo	Nenhuma

2.3 Perfis de usuário (personas)

Marina, 29 · visitante planejando a viagem

Usa o celular, quer ver horários, atividades e comprar ingressos sem se perder. Precisa receber o comprovante no e-mail e abrir o QR Code rapidamente na portaria.

Carlos, 41 · superadmin

Responde pela operação. Precisa ajustar preços, ativar ou desativar tipos de ingresso, acompanhar pedidos e conceder acesso a outras pessoas de forma controlada.

Dev · mantenedor

Sobe o sistema com um comando, cria o primeiro superadmin pelo terminal e precisa de logs, healthcheck e testes confiáveis.

Ana · avaliadora de portfólio

Quer entender rapidamente o que foi construído, como foi decidido e o que ainda é limitação, tudo sustentado por evidências.

2.4 Objetivos e indicadores de sucesso

ID	Objetivo	Indicador	Meta
OB1	Jornada de compra curta	Passos do usuário logado, do clique em Comprar ao QR Code	3 passos (tipo, quantidade e data, confirmar)
OB2	Experiência responsiva	Rolagem horizontal nas telas verificadas	Zero em 390 px e 1280 px
OB3	Carregamento leve	Tamanho publicado do front-end	5,1 MB (reduzido de 36 MB após otimizar imagens)
OB4	Segurança básica sólida	Itens do OWASP Top 10 com controle aplicado	10 de 10 tratados (cap. 12)
OB5	Qualidade verificável	Testes automatizados (API e CLI) e ponta a ponta (interface)	5 de 5 e 12 de 12 aprovados
OB6	Operação simples	Comandos para subir o sistema	1 (docker compose up --build)

2.5 Premissas, restrições e dependências

Tipo	Descrição
Premissa	Os usuários têm navegador moderno e acesso a e-mail.
Premissa	Todos os dados e valores são fictícios; não há tratamento de dados reais de clientes.
Restrição	Sem pagamento real, sem controle de capacidade e sem leitura de QR Code na portaria.
Restrição	Banco SQLite em arquivo: uma única instância da API por volume de dados.

Tipo	Descrição
Restrição	Node.js 22.13 ou superior (uso do módulo nativo node:sqlite).
Dependência	Servidor SMTP (Mailpit na demonstração; SMTP real em produção).
Dependência	Google Maps (iframe) para o mapa de como chegar.
Dependência	Registro npm e Docker Hub para construir as imagens.

2.6 Riscos do projeto

Probabilidade (P) e impacto (I) em escala de 1 (baixo) a 3 (alto); severidade = P × I.

ID	Risco	P	I	Sev.	Resposta
R1	Vazamento de credenciais por segredo fraco ou versionado	2	3	6	JWT_SECRET obrigatório (mín. 32 caracteres) em produção; .env fora do Git
R2	Abuso por força bruta no login	2	2	4	Limite de requisições e mensagens genéricas
R3	Falha do SMTP impedindo comprovantes	2	2	4	Pedido não é perdido; estado pendente e reenvio
R4	Perda de dados do SQLite	1	3	3	Volume persistente e procedimento de backup (cap. 17)
R5	Confusão com um site oficial	2	3	6	Aviso de fã em todas as áreas relevantes (RF05) e neste livro
R6	Token em localStorage exposto por XSS	1	3	3	CSP estrita; evolução recomendada: cookie HttpOnly
R7	Imagens pesadas degradando o carregamento	3	2	6	Mitigado: JPEG otimizado, carregamento sob demanda



PARTE II

Especificação de requisitos

O que o sistema deve fazer: telas, requisitos, regras, casos de uso e dados.

03 · Descrição geral e interfaces

04 · Requisitos funcionais

05 · Requisitos não funcionais

06 · Regras de negócio

07 · Casos de uso

08 · Modelo de dados

CAPÍTULO 3

Descrição geral e interfaces

3.1 Perspectiva do produto

O sistema é uma aplicação web autônoma de três camadas (apresentação, aplicação e dados) acompanhada de uma ferramenta de linha de comando. Ele interage com dois serviços externos: um servidor SMTP, para o envio de comprovantes, e o Google Maps, apenas por *iframe*. A figura abaixo apresenta o **diagrama de contexto** (C4, nível 1).

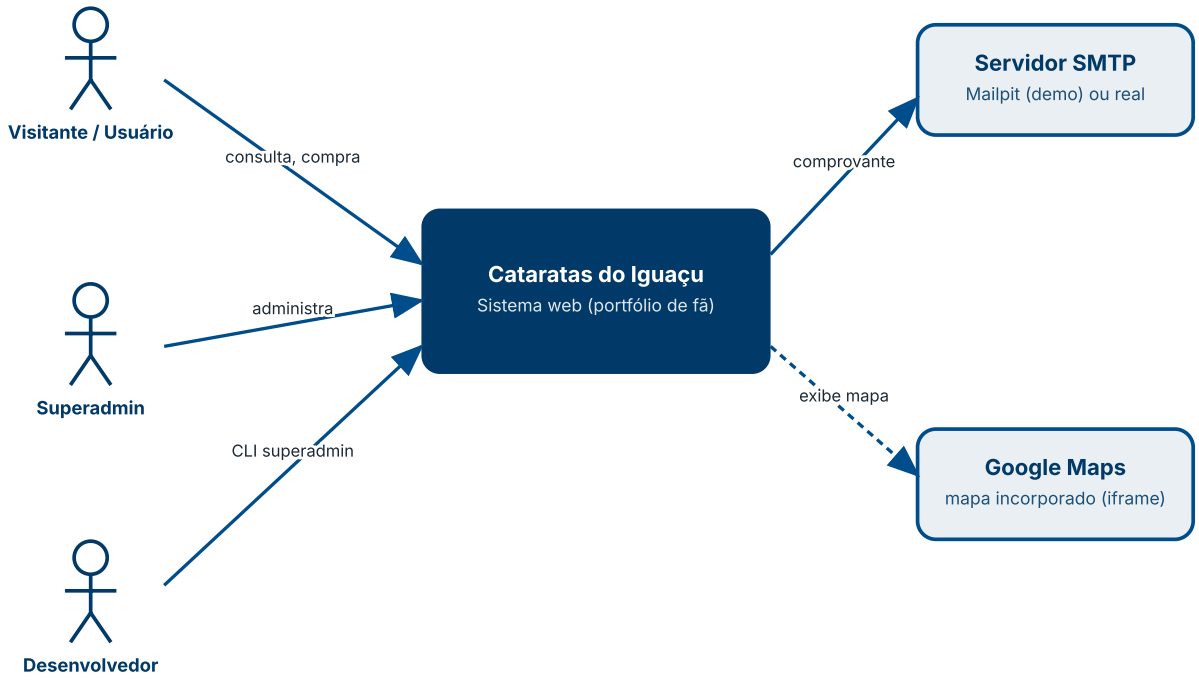


Figura 3.1 · Diagrama de contexto (C4 nível 1).

3.2 Interfaces externas

ID	Interface	Descrição
IE1	Interface de usuário	SPA Angular responsiva (celular a desktop), temas claro e escuro, em português do Brasil.
IE2	API HTTP/JSON	Endpoints REST sob /api, autenticação por Bearer JWT, erros em { erro, campos } (cap. 11).
IE3	SMTP	Envio de e-mail HTML com anexo inline (cid:qrcode). Servidor configurável por variáveis de ambiente.

ID	Interface	Descrição
IE4	Google Maps	Iframe incorporado no painel Como chegar, permitido pela CSP (frame-src).
IE5	Linha de comando	node src/cli.js superadmin <comando> para gestão de superadmins (cap. 14).

3.3 Visão geral das telas

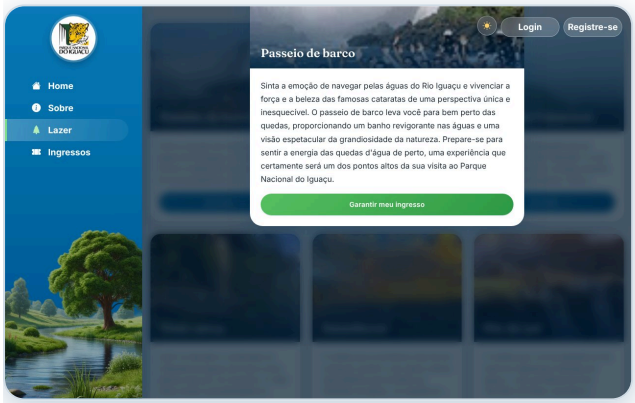
O design mantém a identidade do projeto original: azul das águas, verde da mata, barra lateral com a ilustração do rio no desktop e menu superior no celular. A evolução adicionou tipografia moderna, vidro fosco no hero, cartões com imagem, bilhetes, animações e tema escuro.



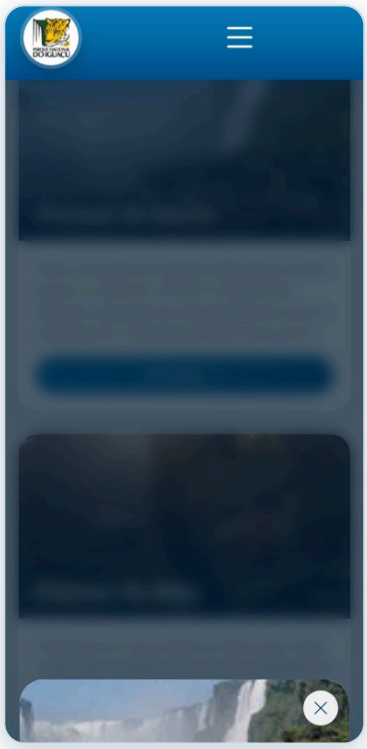
Página inicial: título, chamada para compra e atalhos informativos (modo claro).



Lazer: grade responsiva de cartões com imagem, título e resumo.



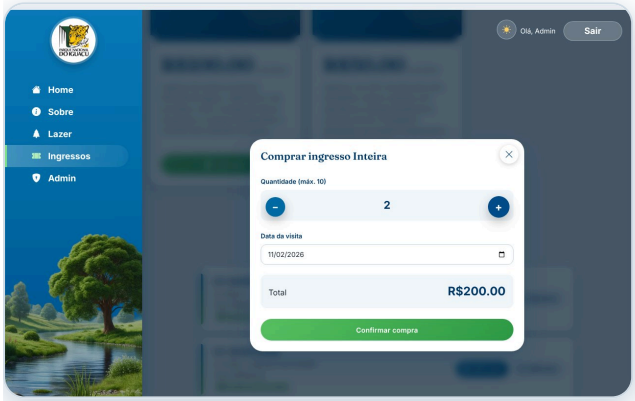
Detalhes de uma atividade no desktop.



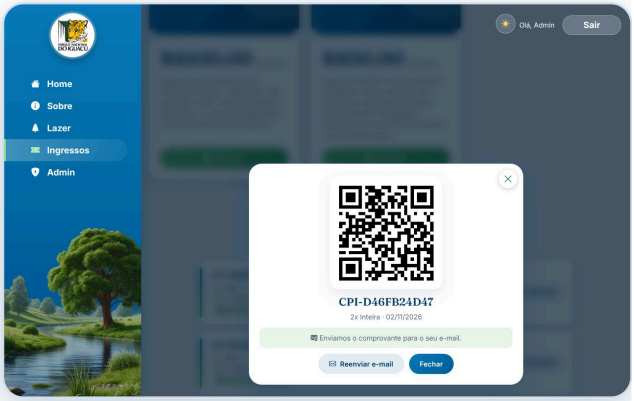
Detalhes no celular.



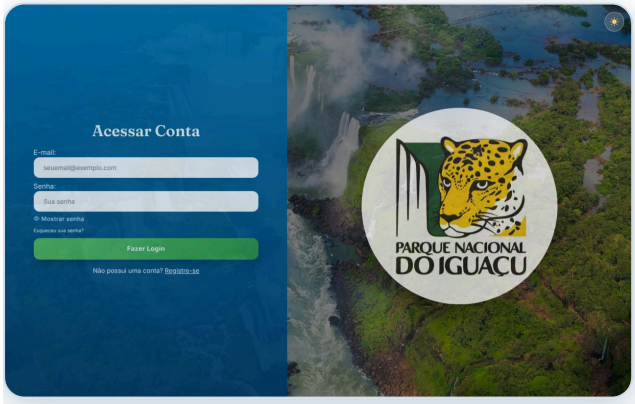
Ingressos: bilhetes e lista de pedidos do usuário.



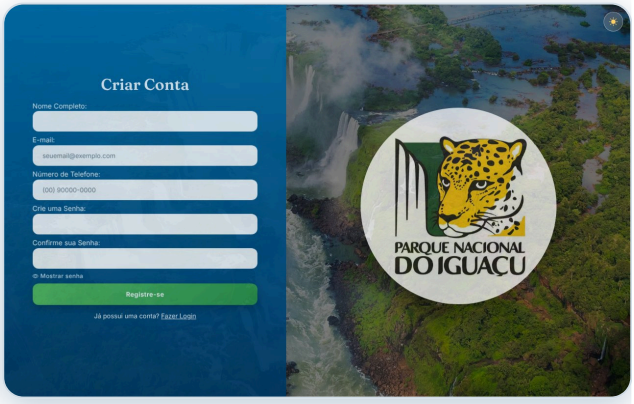
Compra: quantidade, data e total.



Comprovante: QR Code e status do e-mail.



Login.



Cadastro.



Home no celular com menu aberto.



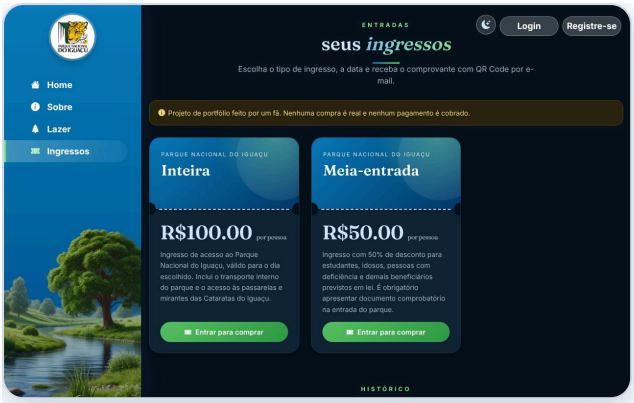
Ingressos e pedidos no celular.



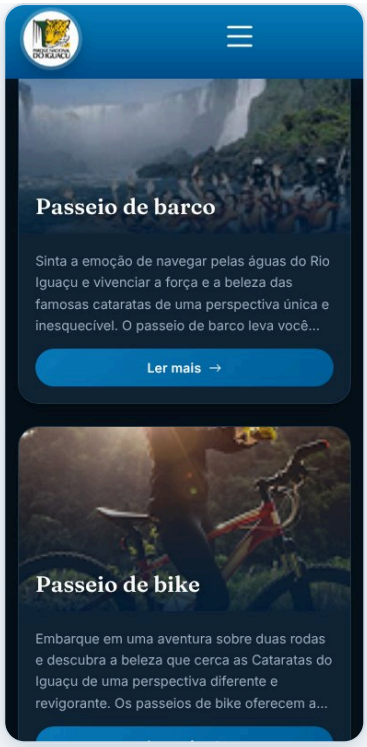
Sobre: texto institucional e aviso de projeto de fã.

3.4 Modo claro e modo escuro

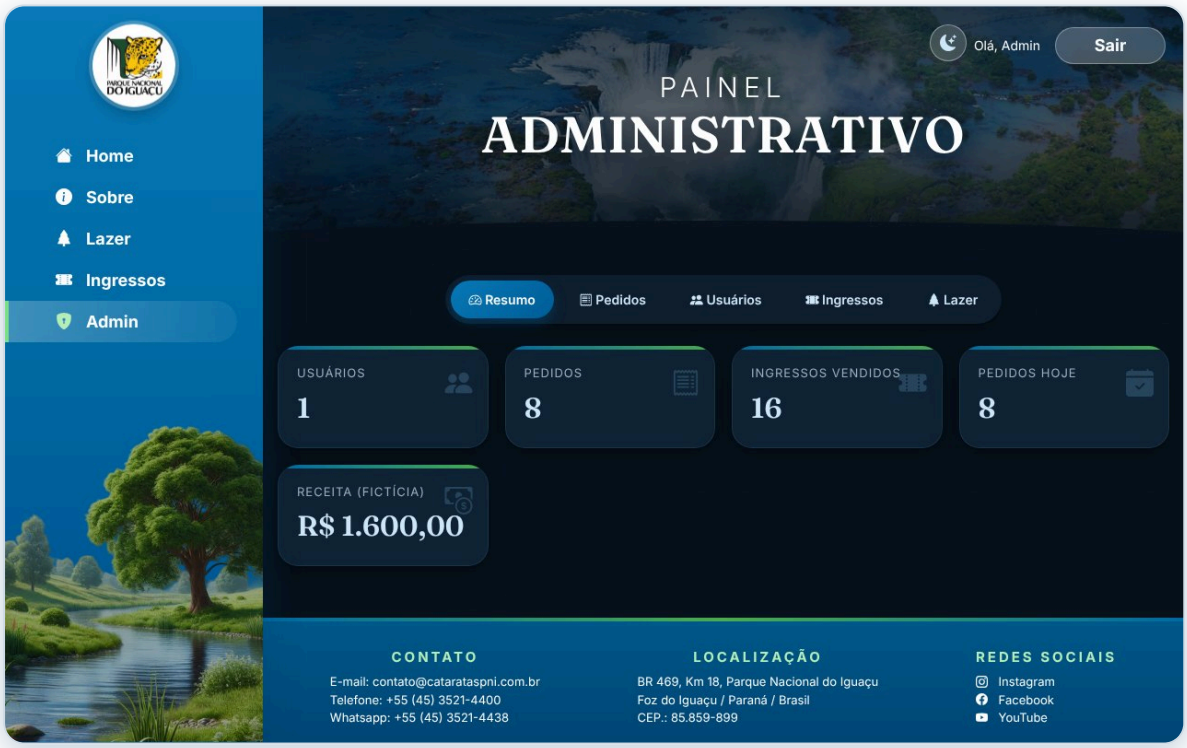
Um botão com ícone de sol e lua alterna o tema. Sem escolha prévia, o sistema segue a preferência do sistema operacional; a escolha manual é guardada no navegador e aplicada antes da primeira renderização, evitando o flash de tema errado.



Ingressos no modo escuro.

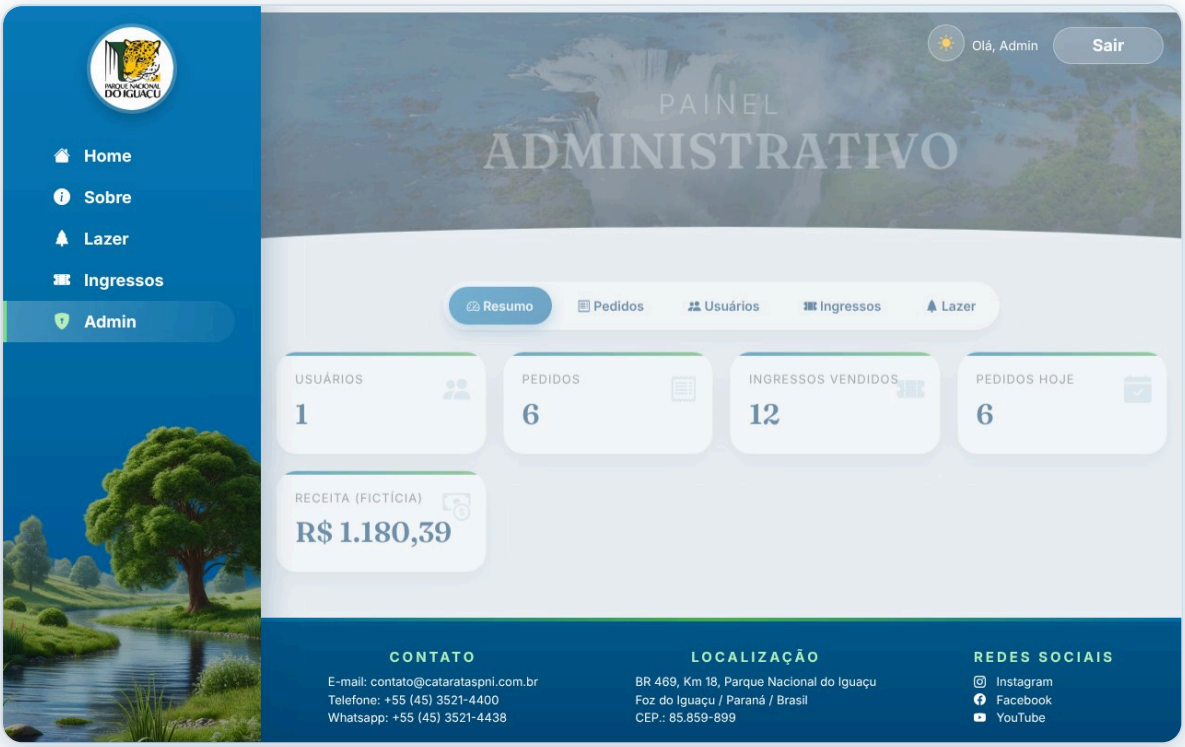


Lazer no modo escuro (celular).

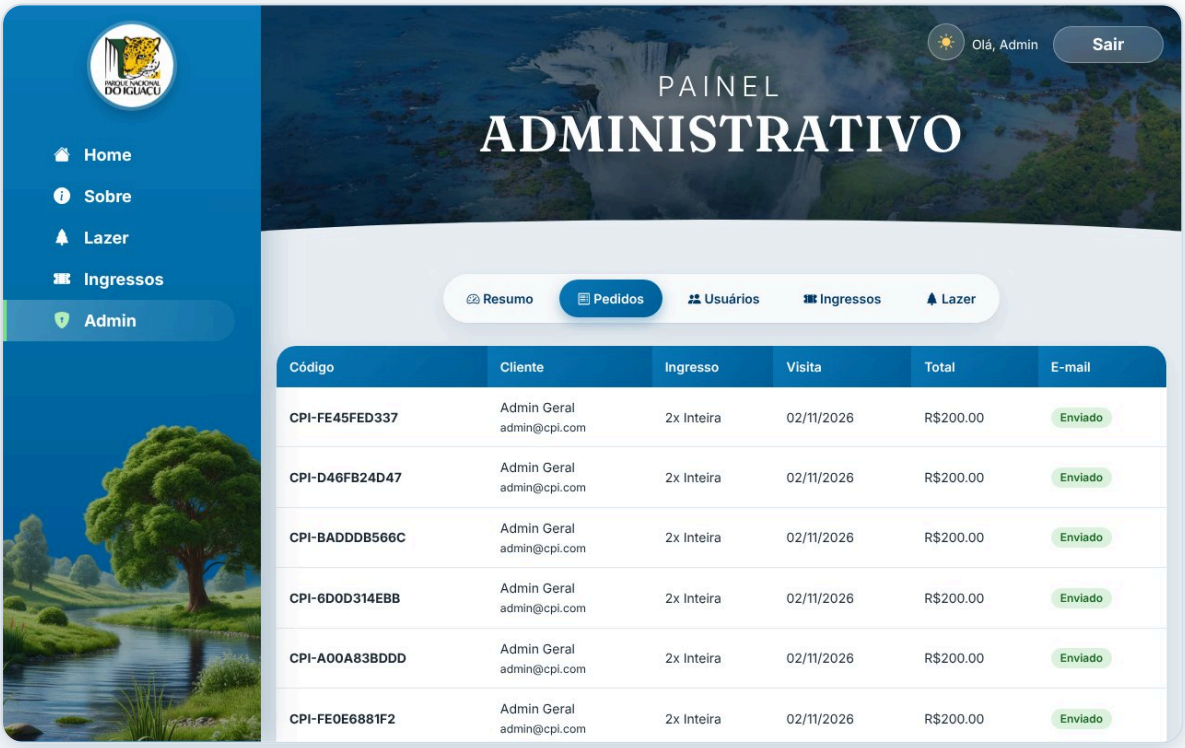


Painel administrativo no modo escuro, com contadores animados.

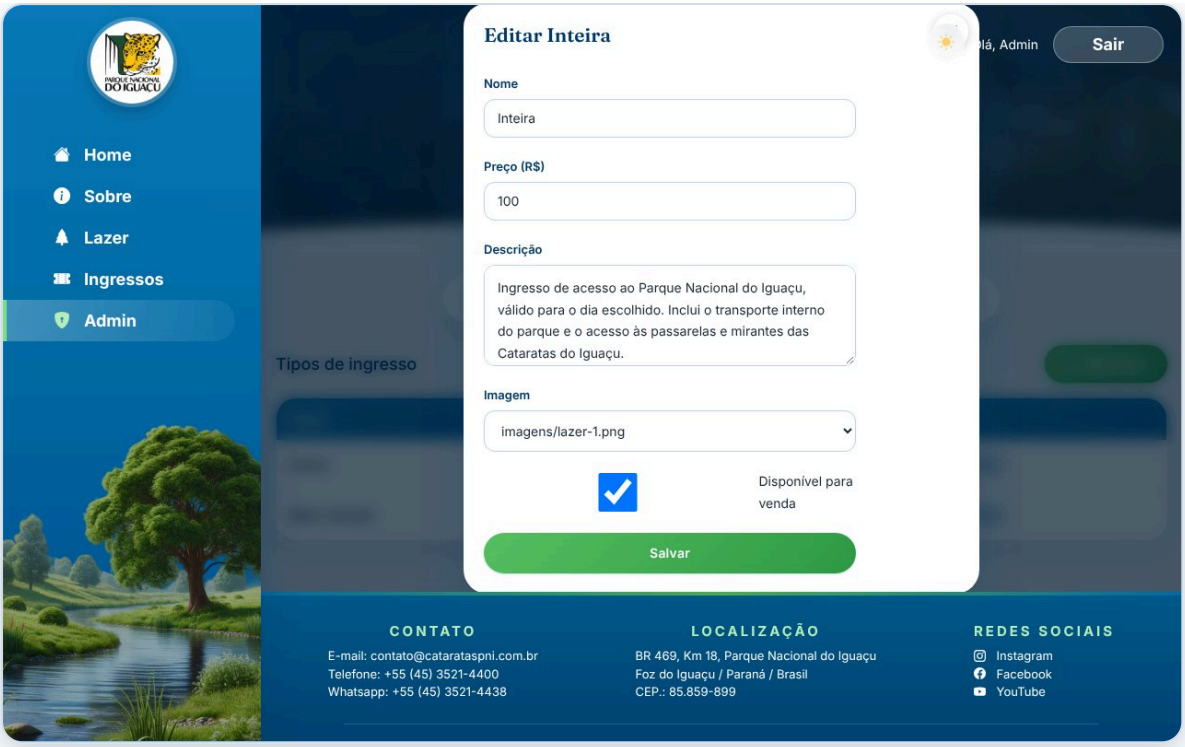
3.5 Painel administrativo



Resumo: métricas gerais.



Pedidos: consulta com status do e-mail.



Edição de um tipo de ingresso.

3.6 Características dos usuários e ambiente

Perfil	Papel no sistema	Acesso
Visitante	Consulta conteúdo público	Sem login
Usuário	Compra e consulta os próprios pedidos	Cadastro e login
Superadmin	Administra ingressos, atividades, pedidos e usuários	Papel superadmin, concedido por outro superadmin ou pela CLI
Desenvolvedor	Opera a infraestrutura e a CLI	Acesso ao terminal / Docker

CAPÍTULO 4

Requisitos funcionais

Os requisitos abaixo são **necessários, verificáveis, não ambíguos e rastreáveis**, conforme a ISO/IEC/IEEE 29148. Cada um possui prioridade MoSCoW, critério de aceitação e vínculo com um caso de uso. A matriz completa de rastreabilidade está no capítulo 16.

Conteúdo público

ID	Requisito	Prior.	Critério de aceitação	UC
RF01	Exibir a página inicial com título, chamada para compra e atalhos de Funcionamento, Como chegar, Estacionamento e Mapa.	Must	*Dado* um visitante na página inicial, *quando* clica em um atalho, *então* vê o painel correspondente e pode fechá-lo.	UC01
RF02	Exibir a página Sobre com a descrição do parque.	Should	*Dado* um visitante, *quando* abre Sobre, *então* lê o texto institucional e o aviso de projeto de fã.	UC01
RF03	Listar as atividades de lazer em cartões e exibir o detalhe completo em janela modal.	Must	*Dado* a lista carregada, *quando* clica em Ler mais, *então* abre o texto completo, fechável por botão, fundo ou Esc.	UC01
RF04	Listar os tipos de ingresso ativos com nome, descrição e preço.	Must	*Dado* tipos ativos e inativos, *quando* abre Ingressos, *então* só os ativos aparecem, com preço em reais.	UC01
RF05	Exibir o aviso de projeto de fã no rodapé e nas páginas Sobre e Ingressos.	Must	*Dado* qualquer página com layout, *então* o rodapé informa que não é o site oficial.	UC01
RF06	Permitir alternar entre tema claro e escuro, respeitando a preferência do sistema e guardando a escolha.	Should	*Dado* sem escolha prévia, *então* o tema segue o sistema; *quando* alterna, *então* persiste ao recarregar.	UC02

Conta e acesso

ID	Requisito	Prior.	Critério de aceitação	UC
RF07	Permitir o cadastro com nome, e-mail, telefone e senha com confirmação.	Must	*Dado* dados válidos, *quando* envia, *então* a conta é criada e a sessão iniciada; e-mail repetido retorna 409.	UC03
RF08	Permitir login por e-mail e senha, mantendo a sessão.	Must	*Dado* credenciais corretas, *então* recebe token; incorretas, *então* 401 com mensagem genérica.	UC04
RF09	Permitir encerrar a sessão.	Must	*Quando* clica em Sair, *então* o token é descartado e o menu volta ao estado público.	UC04

ID	Requisito	Prior.	Critério de aceitação	UC
RF10	Oferecer recuperação de senha com resposta genérica.	Could	*Dado* qualquer e-mail válido, *então* a resposta é a mesma, exista ou não a conta (simulada).	UC04
RF11	Redirecionar visitantes ao login ao tentar comprar e devolvê-los à página de origem.	Should	*Dado* visitante em Ingressos, *quando* clica em comprar e faz login, *então* retorna a Ingressos.	UC05

Compra de ingressos

ID	Requisito	Prior.	Critério de aceitação	UC
RF12	Permitir a compra escolhendo tipo, quantidade (1 a 10) e data da visita.	Must	*Dado* usuário logado e dados válidos, *então* o pedido é criado com total calculado no servidor.	UC05
RF13	Gerar código único (CPI-XXXXXXXXXX) e QR Code por pedido.	Must	*Dado* um pedido, *então* há um código único e um QR Code em PNG.	UC05, UC06
RF14	Enviar o comprovante com QR Code por e-mail.	Must	*Dado* pedido criado, *então* o e-mail é enviado e o pedido marcado como enviado; se falhar, fica pendente.	UC05
RF15	Permitir o reenvio do comprovante.	Should	*Dado* um pedido próprio, *quando* reenvia, *então* novo e-mail é enviado ou erro 502 é informado.	UC06
RF16	Listar os pedidos do usuário com o status do e-mail.	Must	*Dado* usuário logado, *então* vê somente os próprios pedidos, do mais recente ao mais antigo.	UC07

Administração

ID	Requisito	Prior.	Critério de aceitação	UC
RF17	Exibir resumo de usuários, pedidos, ingressos vendidos, pedidos do dia e receita.	Should	*Dado* superadmin, *então* vê as cinco métricas; outros papéis recebem 403.	UC08
RF18	Permitir ao superadmin consultar todos os pedidos.	Must	*Dado* superadmin, *então* lista pedidos de todos, com cliente e status do e-mail.	UC08
RF19	Permitir consultar usuários e alterar o papel (usuário ou superadmin).	Must	*Dado* superadmin, *quando* altera o papel de outro usuário, *então* vale imediatamente; no próprio usuário é bloqueado.	UC08
RF20	Permitir criar e editar tipos de ingresso (nome, preço, descrição, imagem, disponibilidade).	Must	*Dado* superadmin, *quando* salva, *então* a vitrine pública reflete a mudança.	UC09
RF21	Permitir criar, editar e excluir atividades de lazer.	Should	*Dado* superadmin, *quando* salva ou exclui, *então* a página Lazer reflete a mudança.	UC10

Operação

ID	Requisito	Prior.	Critério de aceitação	UC
RF22	Permitir ao desenvolvedor cadastrar e alterar superadmins pelo terminal.	Must	*Dado* acesso ao terminal, *quando* executa a CLI, *então* cria, atualiza, promove, rebaixa ou lista superadmins.	UC11

Situação: os 22 requisitos estão implementados. Os critérios de aceitação são exercitados pelos testes automatizados da API e da CLI (cap. 15) e por verificação scriptada da interface em navegador.

CAPÍTULO 5

Requisitos não funcionais

Os requisitos não funcionais estão classificados pelas características de qualidade da **ISO/IEC 25010**. Cada item traz uma métrica ou evidência observável e o método de verificação, evitando afirmações vagas.

ID	Característica	Requisito	Métrica / evidência	Verificação
RNF01	Desempenho	Pacote inicial pequeno e rotas carregadas sob demanda (lazy loading).	Inicial: 660 kB bruto / 125 kB transferidos (JS + CSS); páginas de 1,3 a 10 kB cada	Saída do ng build
RNF02	Desempenho	Imagens otimizadas; hero em JPEG progressivo.	cataratas-1.jpg: 466 kB (antes 15,8 MB); pasta publicada: 5,1 MB (antes 36 MB)	Inspeção dos arquivos e du
RNF03	Compatibilidade	Funcionar em navegadores modernos.	Verificado em Chromium; Firefox e Safari recentes esperados (CSS e APIs padrão)	Execução scriptada em Chromium
RNF04	Usabilidade	Interface responsiva sem rolagem horizontal.	Zero estouro em 390 px e 1280 px, em todas as telas	Script com navegador headless
RNF05	Usabilidade	Tema claro e escuro, sem flash na carga.	Tema definido por script externo antes do Angular iniciar; persistência em localStorage	Teste de alternância, recarga e prefers-color-scheme
RNF06	Usabilidade	Movimento reduzido respeitado.	Todas as animações desativadas com prefers-reduced-motion	Inspeção do CSS
RNF07	Acessibilidade	Boas práticas de WCAG 2.2: rótulos, foco visível, teclado.	Labels em campos, aria-label em ícones, foco visível, Esc fecha janelas	Inspeção; auditoria formal não realizada
RNF08	Confiabilidade	Resiliência a falhas de e-mail e reinícios.	Pedido preservado se o SMTP falhar; restart unless-stopped; healthcheck a cada 15 s	Teste de reenvio; configuração do compose
RNF09	Segurança	Armazenamento seguro de senhas.	bcrypt custo 12; senha de 8 a 72 caracteres	Teste da CLI e da API; revisão de código

ID	Característica	Requisito	Métrica / evidência	Verificação
RNF10	Segurança	Sessões assinadas e com validade.	JWT HS256 com algoritmo fixado na verificação; validade 7 dias; usuário recarregado do banco a cada requisição	Testes de 401 e de papel
RNF11	Segurança	Validação de entrada e consultas seguras.	Zod em todas as rotas; SQL 100% parametrizado; corpo máximo 50 kB	Testes de 400; revisão de código
RNF12	Segurança	Proteção contra abuso.	300 req/min por IP na API; 50 por 15 min nas rotas de autenticação	Configuração e inspeção
RNF13	Segurança	Cabeçalhos de segurança e CSP restritiva.	CSP default-src self; frame-ancestors none; nosniff; Referrer-Policy no-referrer	Inspeção dos cabeçalhos
RNF14	Segurança	Infraestrutura endurecida.	Usuário não root, back-end somente leitura, cap_drop ALL, no-new-privileges, nginx sem privilégios	Inspeção do compose e dos Dockerfiles
RNF15	Segurança	Segredos obrigatórios em produção.	Sem JWT_SECRET de 32+ caracteres o servidor não inicia	Execução com e sem a variável
RNF16	Manutenibilidade	Código modular, tipado e testado, sem comentários.	Back-end em 55 arquivos de até 58 linhas; front-end Angular standalone em componentes pequenos; 12 testes de API, CLI e unidade e 6 cenários E2E (12 execuções); ESLint nos três projetos	Revisão; npm test e playwright test
RNF17	Portabilidade	Execução reproduzível por contêineres.	docker compose up --build sobe frontend, backend e mailpit	Configuração do compose (build não validado no sandbox)
RNF18	Operabilidade	Diagnóstico simples.	GET /api/health; logs no stdout; erros 500 registrados sem vazar detalhes ao cliente	Chamada ao endpoint e inspeção do código
RNF19	Privacidade	Minimização e finalidade (LGPD).	Coleta apenas nome, e-mail, telefone e senha (hash); senhas nunca retornam pela API; dados fictícios	Revisão das rotas e do esquema

As métricas de tamanho foram obtidas do próprio build. Itens marcados como inspeção não substituem testes de carga ou auditorias formais de acessibilidade e segurança, que ficam como evolução (cap. 19).

CAPÍTULO 6

Regras de negócio

Regras de negócio são políticas do domínio que restringem ou orientam os requisitos. Cada regra traz sua **justificativa** e os requisitos relacionados.

ID	Regra	Justificativa	Relacionados
RN01	O e-mail é único no sistema e normalizado para minúsculas.	Evitar contas duplicadas e ambiguidade no login.	RF07, RF08
RN02	A senha deve ter de 8 a 72 caracteres.	Resistência mínima a adivinhação; 72 bytes é o limite do bcrypt.	RF07
RN03	O telefone deve conter 10 ou 11 dígitos, com DDD.	Padrão telefônico brasileiro.	RF07
RN04	Cada pedido comporta de 1 a 10 ingressos de um mesmo tipo.	Limitar abuso e simplificar o comprovante.	RF12
RN05	A data da visita não pode estar no passado (tolera-se o dia anterior em UTC).	Evitar compras inválidas sem rejeitar usuários de fusos diferentes.	RF12
RN06	O total é calculado no servidor: preço atual do tipo × quantidade.	Integridade financeira; o cliente não define valores.	RF12
RN07	O pedido guarda o nome do tipo no momento da compra.	Preservar o histórico se o tipo for renomeado.	RF12, RF16
RN08	Somente tipos de ingresso ativos podem ser comprados.	Permitir retirar tipos da vitrine sem apagar histórico.	RF04, RF12, RF20
RN09	Meia-entrada exige documento comprobatório na entrada.	Regra do benefício; informativa no sistema.	RF04
RN10	O código do pedido segue CPI- + 10 caracteres hexadecimais maiúsculos.	Identificador curto, único e legível.	RF13
RN11	Um usuário só consulta, reenvia e gera QR Code dos próprios pedidos.	Confidencialidade (controle de acesso por objeto).	RF15, RF16
RN12	O cadastro público sempre cria usuários comuns.	Impedir escalada de privilégio.	RF07
RN13	Um superadmin não altera o próprio papel pelo painel.	Evitar perda acidental de acesso administrativo.	RF19
RN14	A CLI não rebaixa o único superadmin sem --forcar.	Evitar ficar sem administrador.	RF22

ID	Regra	Justificativa	Relacionados
RN15	O parque funciona todos os dias: seg-sex 9h-16h; sáb-dom 8h30-16h.	Informação institucional exibida ao visitante.	RF01
RN16	Falha no e-mail não cancela o pedido; ele fica pendente e pode ser reenviado.	Disponibilidade da compra independente do SMTP.	RF14, RF15
RN17	Valores, pagamentos e e-mails são fictícios e sem validade real.	Natureza de portfólio.	RF05
RN18	O sistema deve sempre informar que é homenagem de fã.	Evitar confusão com site oficial (risco R5).	RF05
RN19	Sem escolha prévia, o tema segue o sistema; a escolha manual prevalece.	Respeitar preferência do usuário.	RF06

CAPÍTULO 7

Casos de uso

O diagrama abaixo (UML) apresenta os atores e os casos de uso. A seguir, os casos principais são especificados com pré-condições, fluxos e pós-condições.

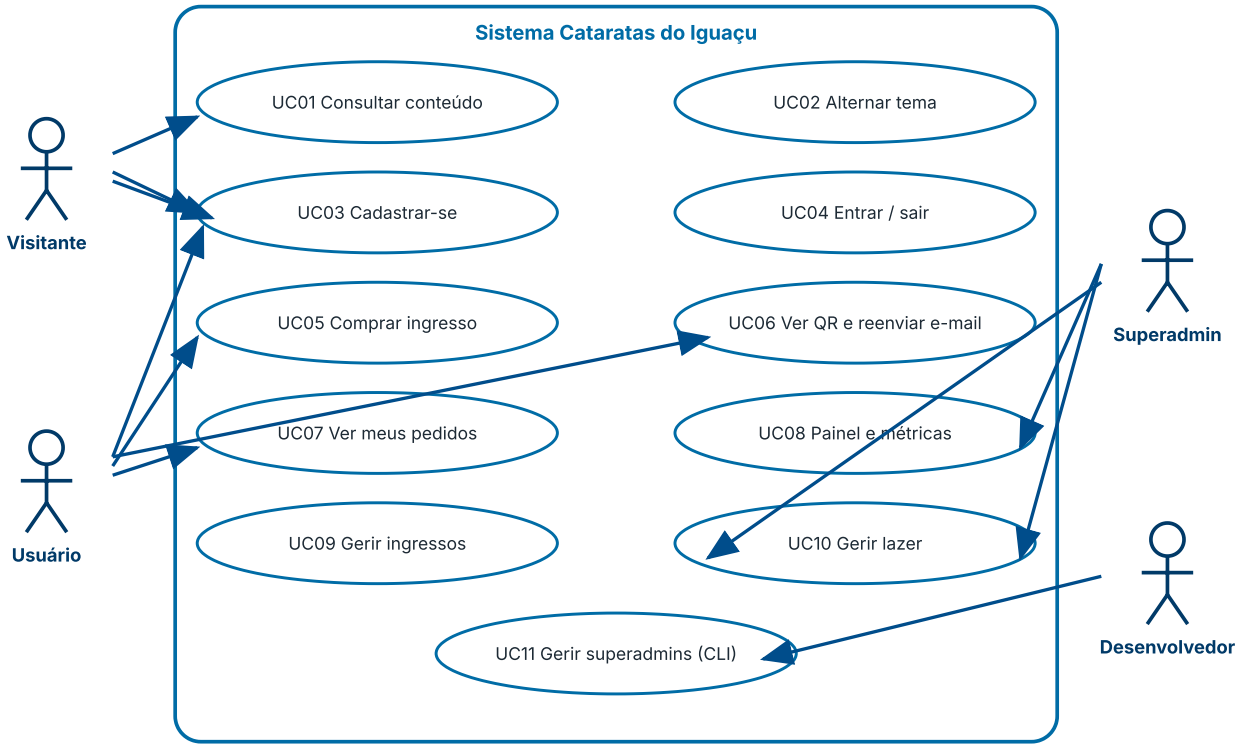


Figura 7.1 · Diagrama de casos de uso (UML).

7.1 Catálogo

ID	Caso de uso	Ator	RF
UC01	Consultar conteúdo (home, sobre, lazer, ingressos)	Visitante	RF01-RF05
UC02	Alternar tema	Visitante, Usuário	RF06
UC03	Cadastrar-se	Visitante	RF07
UC04	Entrar e sair; recuperar senha	Visitante, Usuário	RF08-RF10
UC05	Comprar ingresso	Usuário	RF11-RF14
UC06	Ver QR Code e reenviar e-mail	Usuário	RF13, RF15
UC07	Ver meus pedidos	Usuário	RF16

ID	Caso de uso	Ator	RF
UC08	Painel, métricas, pedidos e usuários	Superadmin	RF17-RF19
UC09	Gerir tipos de ingresso	Superadmin	RF20
UC10	Gerir atividades de lazer	Superadmin	RF21
UC11	Gerir superadmins pela CLI	Desenvolvedor	RF22

7.2 Especificações

UC03 Cadastrar-se	
Ator	Visitante
Pré-condições	Não autenticado; e-mail ainda não cadastrado.
Fluxo principal	1. Abre Registre-se e preenche nome, e-mail, telefone, senha e confirmação. 2. O front valida os campos e envia POST /api/auth/registro. 3. A API valida (Zod), normaliza o e-mail e aplica bcrypt à senha. 4. A API cria o usuário com papel usuario e devolve token e perfil. 5. O sistema inicia a sessão e redireciona ao destino seguro.
Fluxos alternativos e exceções	• E-mail já cadastrado: 409 "Já existe uma conta com este e-mail". • Dados inválidos (senha curta, telefone inválido, senhas diferentes): 400 com a mensagem do primeiro erro. • Limite de requisições excedido: 429.
Pós-condições	Usuário criado com papel usuario e sessão ativa.
Rastreio	RF07; RN01, RN02, RN03, RN12

UC05 Comprar ingresso	
Ator	Usuário
Pré-condições	Autenticado; ao menos um tipo ativo.
Fluxo principal	1. Em Ingressos, escolhe um tipo e clica em Comprar. 2. Define a quantidade (1 a 10) e a data da visita; o total é exibido. 3. Confirma; o front envia POST /api/pedidos com token. 4. A API valida, calcula o total, gera o código CPI-XXXXXXXXXX e grava o pedido. 5. A API gera o QR Code e envia o e-mail de comprovante. 6. O front exibe o QR Code, o status do e-mail e um aviso de confirmação.
Fluxos alternativos e exceções	• Visitante sem login: redirecionado ao login e devolvido a Ingressos (RF11). • Tipo inexistente ou inativo: 400. • Data no passado: 400. • Falha do SMTP: pedido mantido, status pendente e opção de reenviar (RN16).
Pós-condições	Pedido persistido; e-mail enviado ou pendente.
Rastreio	RF11-RF14; RN04-RN08, RN10, RN16

UC06 Ver QR Code e reenviar e-mail	
Ator	Usuário
Pré-condições	Pedido próprio existente.
Fluxo principal	1. Em Meus pedidos, clica em QR Code para abrir o comprovante. 2. Clica em Reenviar e-mail. 3. A API envia novamente e atualiza o status. 4. O front mostra um aviso de sucesso.
Fluxos alternativos e exceções	• Pedido de outro usuário ou inexistente: 404. • SMTP indisponível: 502 com mensagem para tentar mais tarde.
Pós-condições	Status do e-mail atualizado quando o envio tem sucesso.
Rastreio	RF13, RF15; RN11, RN16

UC09 Gerir tipos de ingresso	
Ator	Superadmin
Pré-condições	Autenticado com papel superadmin.
Fluxo principal	1. Abre Admin → Ingressos e escolhe Novo tipo ou Editar. 2. Preenche nome, preço em reais, descrição, imagem e disponibilidade. 3. Salva; o front envia POST ou PUT para /api/admin/tipos. 4. A API valida e persiste; a vitrine pública reflete a mudança.
Fluxos alternativos e exceções	• Usuário comum: 403. • Preço negativo, imagem fora do padrão ou textos curtos: 400. • Tipo inativo deixa de ser vendável sem apagar pedidos antigos (RN08).
Pós-condições	Tipo criado ou atualizado.
Rastreio	RF20; RN08

UC11 Gerir superadmins pela CLI	
Ator	Desenvolvedor
Pré-condições	Acesso ao terminal do back-end e ao arquivo de dados.
Fluxo principal	1. Executa node src/cli.js superadmin criar --email ... --nome .... 2. A CLI solicita a senha de forma oculta se --senha não for informada. 3. A CLI valida e grava (ou promove o usuário existente). 4. Usa atualizar, promover, rebaixar ou listar quando necessário.
Fluxos alternativos e exceções	• Senha fora de 8 a 72 caracteres: erro. • E-mail já usado ao atualizar: erro. • Rebaixar o único superadmin sem --forcar: erro (RN14).
Pós-condições	Superadmin criado ou alterado; efeito imediato na API.
Rastreio	RF22; RN12, RN14

CAPÍTULO 8

Modelo de dados

8.1 Diagrama entidade-relacionamento

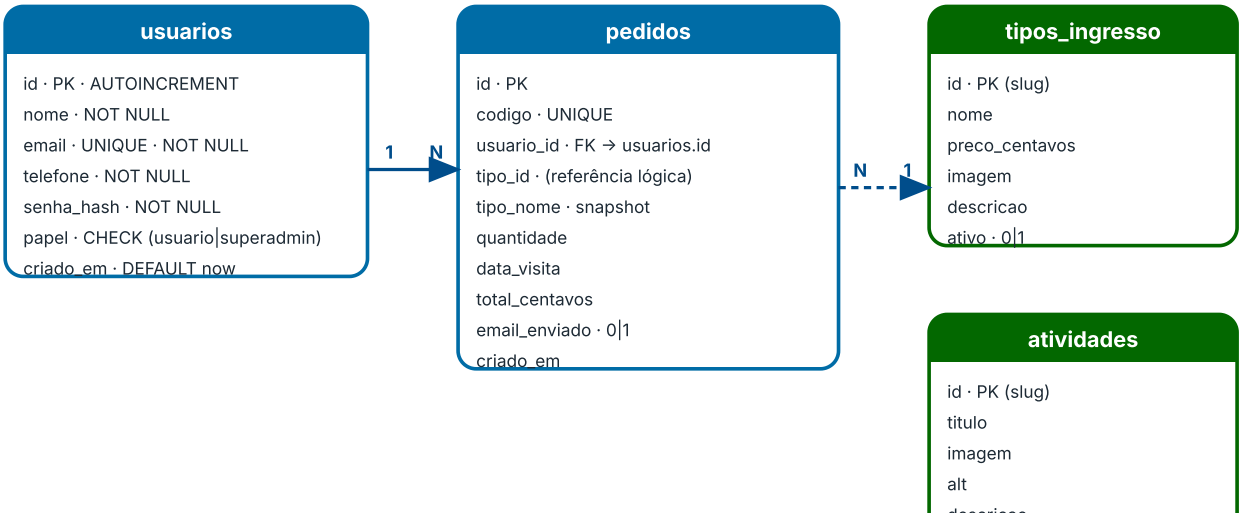


Figura 8.1 · Modelo entidade-relacionamento.

8.2 Dicionário de dados

usuarios

Coluna	Tipo	Restrições	Descrição
id	INTEGER	PK, autoincremento	Identificador
nome	TEXT	NOT NULL	Nome completo (mín. 3 caracteres na entrada)
email	TEXT	NOT NULL, UNIQUE	E-mail em minúsculas (RN01)
telefone	TEXT	NOT NULL, padrão vazio	10 ou 11 dígitos (RN03)
senha_hash	TEXT	NOT NULL	Hash bcrypt (custo 12)
papel	TEXT	NOT NULL, CHECK (usuario|superadmin)	Perfil de acesso
criado_em	TEXT	NOT NULL, padrão CURRENT_TIMESTAMP	Data de criação (UTC)

pedidos

Coluna	Tipo	Restrições	Descrição
id	INTEGER	PK	Identificador
codigo	TEXT	NOT NULL, UNIQUE	CPI- + 10 hexadecimais (RN10)
usuario_id	INTEGER	NOT NULL, FK → usuarios.id	Dono do pedido
tipo_id	TEXT	NOT NULL	Slug do tipo (referência lógica)
tipo_nome	TEXT	NOT NULL	Nome do tipo na data da compra (RN07)
quantidade	INTEGER	NOT NULL	1 a 10 (RN04, validado na API)
data_visita	TEXT	NOT NULL	AAAA-MM-DD
total_centavos	INTEGER	NOT NULL	Total calculado no servidor (RN06)
email_enviado	INTEGER	NOT NULL, padrão 0	0 pendente, 1 enviado (RN16)
criado_em	TEXT	NOT NULL, padrão CURRENT_TIMESTAMP	Data de criação

tipos_ingresso e atividades

Tabela.coluna	Tipo	Descrição
tipos_ingresso.id	TEXT PK	Slug gerado a partir do nome
tipos_ingresso.nome / descricao	TEXT	Textos exibidos na vitrine
tipos_ingresso.preco_centavos	INTEGER	Preço em centavos (0 a 1.000.000)
tipos_ingresso.imagem	TEXT	Caminho imagens/arquivo
tipos_ingresso.ativo	INTEGER	1 vendável, 0 oculto (RN08)
atividades.id	TEXT PK	Slug do título
atividades.titulo / descricao / alt	TEXT	Conteúdo do cartão e texto alternativo
atividades.imagem	TEXT	Caminho imagens/arquivo
atividades.ordem	INTEGER	Ordem de exibição

8.3 Ciclo de vida do envio do comprovante

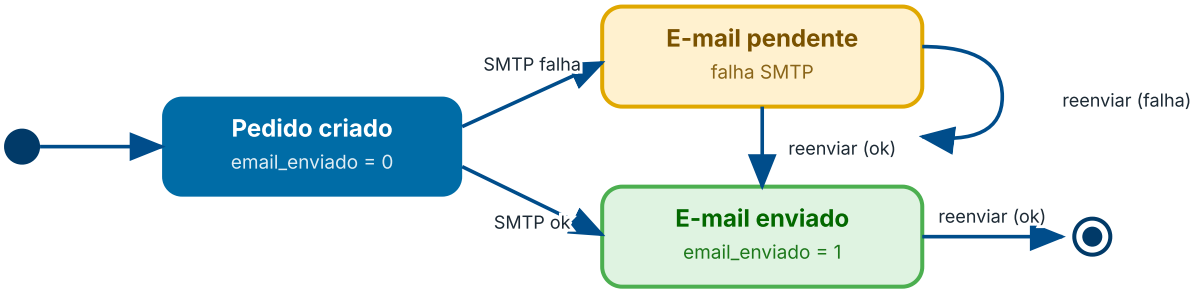


Figura 8.2 · Estados do envio de e-mail de um pedido.

8.4 Decisões de modelagem

- **Dinheiro em centavos inteiros:** elimina erros de ponto flutuante.
- **tipo_nome no pedido:** desacopla o histórico de edições futuras do catálogo.
- **tipo_id sem chave estrangeira:** tipos nunca são removidos, apenas desativados.
- **Semente condicional:** dados iniciais só entram se a tabela estiver vazia, preservando edições do admin.
- **Persistência:** arquivo cataratas.db no volume dados, em modo WAL com chaves estrangeiras ativas.



PARTE III

Projeto e implementação

Como o sistema é construído: arquitetura, front-end, API, segurança, e-mail e administração.

09 · Arquitetura

10 · Front-end Angular

11 · Back-end e API

12 · Segurança

13 · E-mail e comprovante

14 · Administração e CLI

CAPÍTULO 9

Arquitetura

A arquitetura é descrita com o **modelo C4** (contexto, contêineres, componentes) e uma visão de implantação, atendendo ao princípio da ISO/IEC/IEEE 42010 de apresentar o sistema por *visões* voltadas às preocupações de cada parte interessada.

9.1 Contêineres (C4 nível 2)

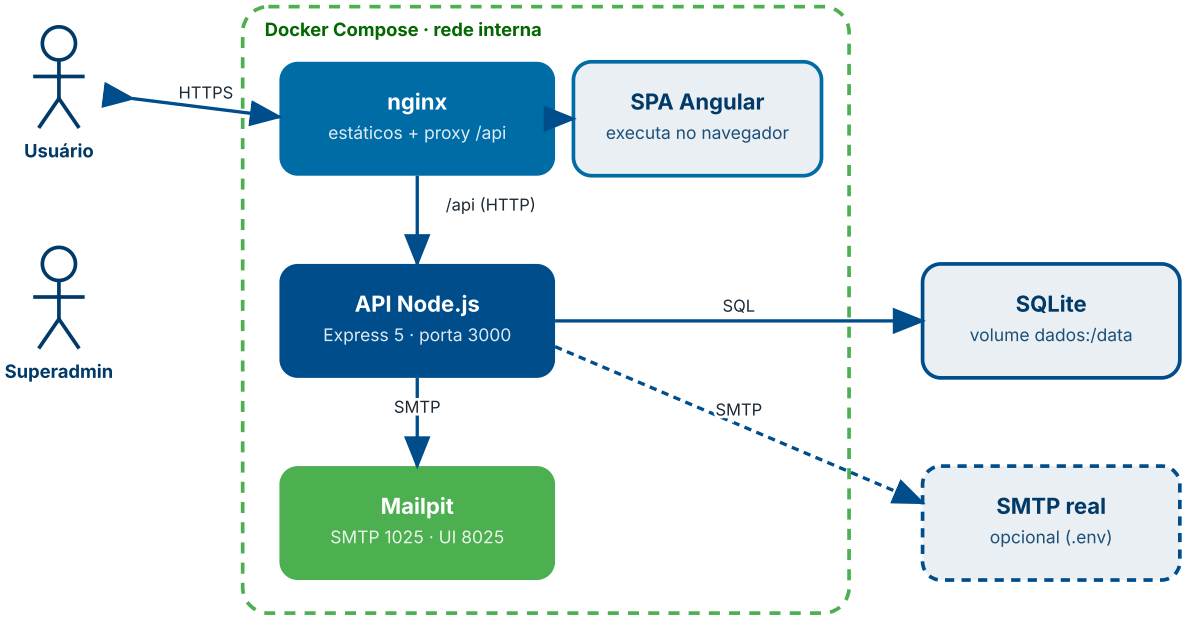


Figura 9.1 · Diagrama de contêineres.

Contêiner	Tecnologia	Responsabilidade
nginx	nginx-unprivileged 1.27	Entrega os estáticos do Angular, aplica cabeçalhos de segurança e encaminha /api.
SPA Angular	Angular 22, Bootstrap 5	Interface, formulários, estado de sessão e tema, guarda de rota do painel.
API Node.js	Node 22, Express 5	Regras de negócio, autenticação, API REST, QR Code, e-mail.
SQLite	node:sqlite	Persistência em arquivo no volume dados.
Mailpit	axllent/mailpit	Caixa de entrada de demonstração (UI na porta 8025).

9.2 Componentes da API (C4 nível 3)

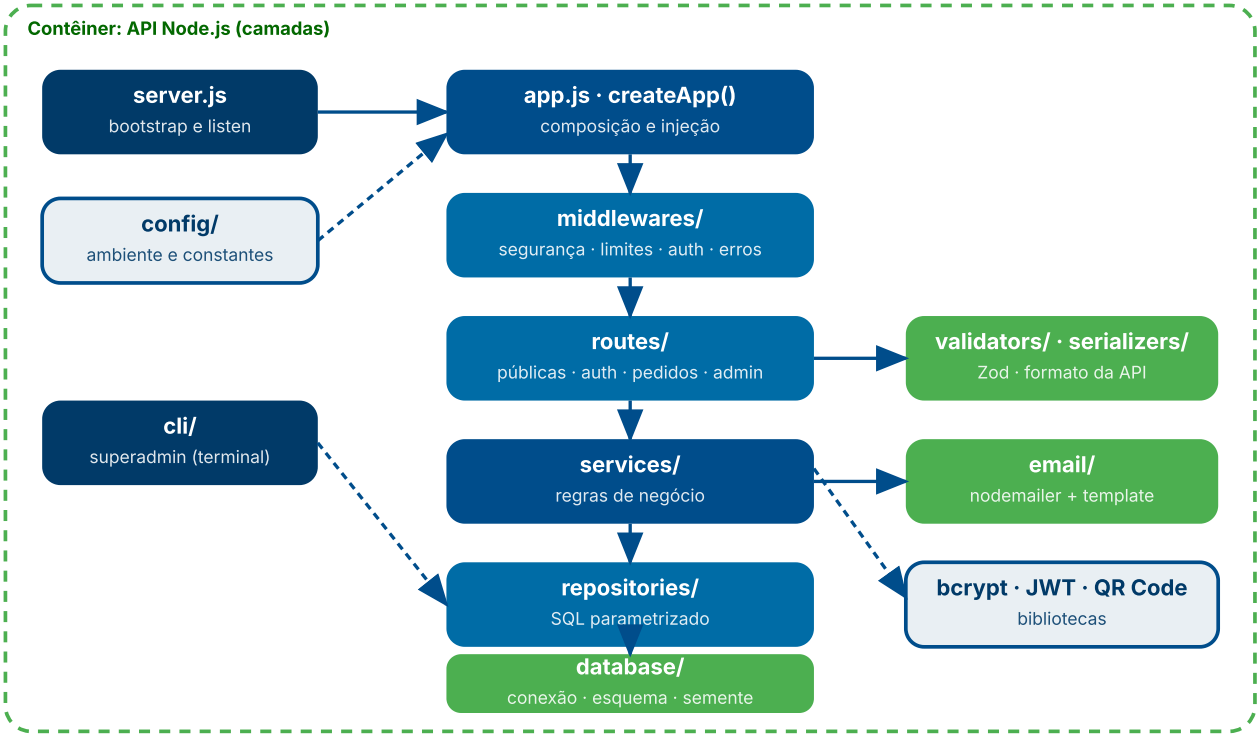


Figura 9.2 · Componentes do back-end.

A API segue o fluxo **rota → serviço → repositório**. As *rotas* só traduzem HTTP (validam a entrada, chamam o serviço e escolhem o código de resposta); os *serviços* concentram as regras de negócio e não conhecem Express; os *repositórios* são a única camada que escreve SQL. *Validators* (Zod) e *serializers* isolam o contrato da API do esquema do banco. A função `createApp({ db, jwtSecret, mailer, staticDir, rateLimitMax })` compõe as camadas por **injeção de dependências**, o que permite testar com banco em memória e e-mail simulado, sem rede. O `server.js` apenas lê o ambiente (`config/`) e liga as peças reais; a CLI reutiliza repositórios e o serviço de senhas.

9.3 Visão de implantação

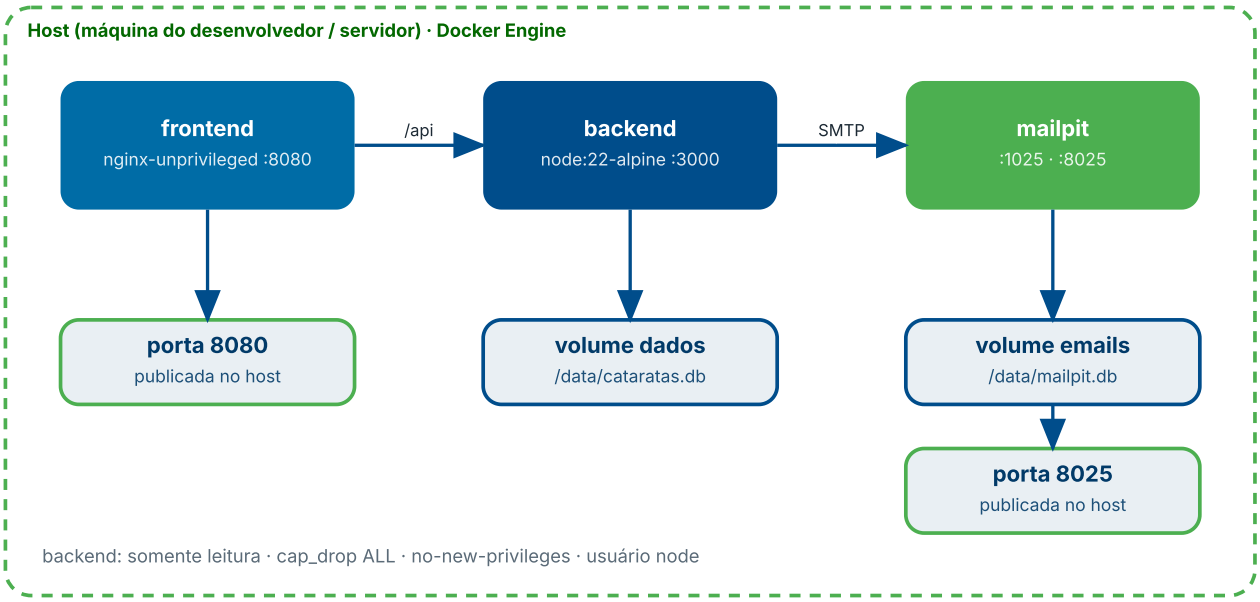


Figura 9.3 · Implantação com Docker Compose.

9.4 Fluxo de uma compra

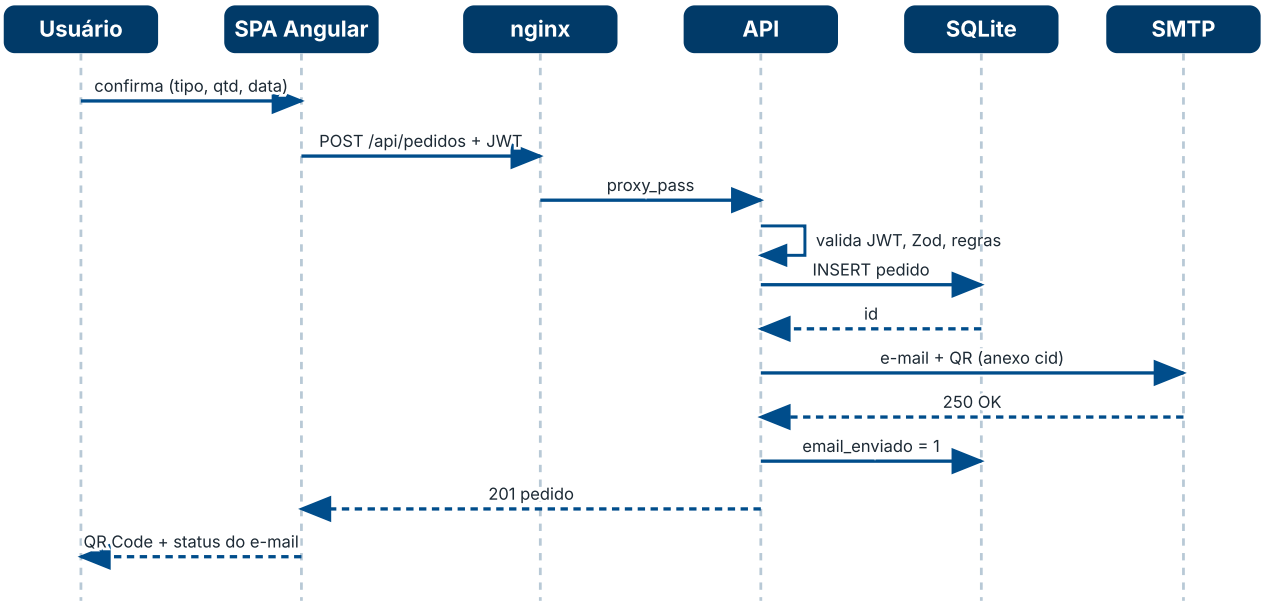


Figura 9.4 · Diagrama de sequência da compra (UC05).

9.5 Decisões arquiteturais (ADR)

ID	Decisão	Contexto / motivo	Alternativas	Consequências
ADR-01	Angular standalone com sinais	Conversão do projeto HTML/Bootstrap para Angular, com estado local simples.	React, Vue	Menos boilerplate (sem NgModules), rotas com lazy loading, tipagem forte.

ID	Decisão	Contexto / motivo	Alternativas	Consequências
ADR-02	Express 5 no back-end	API pequena, ecossistema maduro, requisito de Node.js.	Fastify, NestJS	Composição simples via createApp; menos estrutura imposta.
ADR-03	SQLite via node:sqlite	Portfólio sem serviço de banco externo; execução simples.	PostgreSQL, better-sqlite3	Zero dependência nativa; limita a uma instância; migrar para PostgreSQL é evolução prevista.
ADR-04	Sessão com JWT (HS256, 7 dias)	API stateless e simples de consumir pela SPA.	Sessão em servidor, cookie HttpOnly	Token em localStorage (risco R6); usuário recarregado do banco para refletir mudanças de papel.
ADR-05	Validação com Zod na borda	Uma fonte de verdade de regras, mensagens claras.	Validação manual, Joi	Erros 400 padronizados com campos.
ADR-06	Mesmo domínio via nginx	Front e API na mesma origem evitam CORS e simplificam a CSP.	Domínios separados	Um único ponto de entrada; API não exposta diretamente.
ADR-07	Mailpit como SMTP de demonstração	Mostrar e-mails sem enviar nada à internet.	Mailhog, serviço real	Troca por SMTP real apenas com variáveis de ambiente.
ADR-08	Contêineres endurecidos	Reduzir superfície de ataque.	Contêineres padrão	Back-end somente leitura; exige volume para /data.
ADR-09	Dinheiro em centavos	Exatidão financeira.	Decimal em ponto flutuante	Conversão apenas na camada de apresentação.
ADR-10	CLI para superadmin	Não existir credencial padrão nem rota pública de promoção.	Usuário seed fixo, variável de ambiente	Operação exige acesso ao terminal; senha oculta no prompt.
ADR-11	Back-end em camadas (rotas, serviços, repositórios)	Evitar um arquivo monolítico e permitir testar regras sem HTTP e SQL sem regras.	Arquivo único, framework opinativo (NestJS)	Mais arquivos pequenos (o maior tem 58 linhas); dependências explícitas e testáveis.
ADR-12	Componentes de apresentação e estilos por responsabilidade	Páginas grandes e CSS global duplicado dificultavam a manutenção.	Componente único por página, CSS em um arquivo	Páginas orquestram; componentes pequenos recebem entradas e emitem eventos; um seletor, uma definição.

9.6 Estrutura do repositório

```
site-cataratas-do-iguacu/
├── docker-compose.yml   .env.example
├── backend/   Dockerfile  eslint.config.js
│   ├── src/
│   │   ├── server.js  app.js  cli.js
│   │   ├── config/        ambiente e constantes
│   │   ├── database/      conexão, esquema e semente
│   │   ├── repositories/  SQL (usuários, pedidos, tipos, atividades, métricas)
│   │   ├── services/      regras de negócio
│   │   ├── validators/    esquemas Zod     serializers/  formato da API
│   │   ├── middlewares/   segurança, limites, autenticação, erros
│   │   ├── routes/        públicas, auth, pedidos, admin
│   │   ├── email/         transporte e template     cli/  comandos
│   │   └── shared/        erros HTTP, validação, datas, slug
│   └── test/  api · cli · unit
├── frontend/  Dockerfile  nginx.conf  eslint.config.js
│   ├── public/  imagens/  tema-init.js
│   └── src/   styles/ (13 arquivos por responsabilidade)   legacy/
│       └── app/  core/ (models, services, interceptors, guards, utils)
│                 shared/ (components, directives)   layout/ (nav, footer)
│                 pages/ (home, sobre, lazer, ingressos, login, registro, admin)
├── e2e/       playwright.config.js  tests/fluxos.spec.js
├── .github/workflows/ci.yml
└── docs/livro/  fontes e PDF deste documento
```

9.7 Convenções de código e qualidade estática

Prática	Como é aplicada
Sem comentários no código	Nomes expressivos e funções pequenas substituem comentários; o ESLint falha a análise se houver comentário de linha ou TODO (no-inline-comments, no-warning-comments).
Responsabilidade única	Um arquivo por responsabilidade: o maior arquivo do back-end tem 58 linhas e o do front-end, 97.
Camadas e dependências	Rotas não acessam o banco; serviços não conhecem HTTP; repositórios são o único ponto de SQL.
Injeção de dependências	createApp e os serviços recebem suas dependências, permitindo testes com banco em memória e e-mail simulado.
Constantes nomeadas	Limites, prazos, custos e políticas em config/constants.js e constantes de módulo, sem números mágicos.
Erros tipados	HttpError com fábricas (badRequest, notFound, conflict...) e um único tratador de erros.
Componentes pequenos (front-end)	Páginas orquestram; componentes de apresentação recebem entradas e emitem eventos; template em arquivo próprio.
CSS sem duplicação	Cada seletor tem uma única definição; 13 arquivos por responsabilidade e variáveis para tema.

Prática	Como é aplicada
Análise estática	ESLint no back-end, no front-end (com regras de acessibilidade de template) e no E2E, executado no CI.
Regressão visual	Comparação pixel a pixel de 56 capturas antes e depois da refatoração do front-end: zero diferenças.

CAPÍTULO 10

Front-end Angular

10.1 Organização

O front-end usa **componentes standalone**, **sinais** (`signal` , `computed`) para estado local e **formulários reativos**. As páginas são carregadas sob demanda por `loadComponent` , mantendo o pacote inicial enxuto (RNF01).

Rota	Componente	Acesso
/	Home	Público
/sobre	Sobre	Público
/lazer	Lazer	Público
/ingressos	Ingressos	Público (compra exige login)
/login, /registre-se	Login, Registro	Público, sem menu lateral
/admin	Admin	Superadmin (adminGuard)

10.2 Organização em camadas

Pasta	Conteúdo
core/models	Interfaces TypeScript do contrato da API.
core/services	Api (HTTP tipado), Auth (sessão), Tema, Toast e Redirecionamento.
core/interceptors, guards, utils	Interceptor do JWT, guarda do painel e utilitários (armazenamento, erros, formulários, redirecionamento seguro).
shared/components	Modal, toasts, botão de tema, alternar senha e hero do parque, todos reutilizáveis.
shared/directives	Revelação ao rolar e contador animado.
layout	Menu lateral/superior e rodapé.
pages	Uma pasta por rota; Ingressos e Admin se dividem em componentes menores (bilhete, lista de pedidos, modais de compra e comprovante; abas de resumo, pedidos, usuários, ingressos e lazer).

10.3 Módulos centrais

Arquivo	Função
core/services/api.ts	Serviço HTTP tipado com todos os endpoints e as interfaces de dados.
core/services/auth.ts	Sessão em sinais, login, cadastro, logout e atualização do perfil.
core/interceptors/auth.interceptor.ts	Anexa o JWT às chamadas /api e encerra a sessão em respostas 401.
core/guards/admin.guard.ts, core/utils/safe-redirect.ts	Guarda do painel e destinoSeguro (impede redirecionamento aberto).
shared/components/modal	Janela modal acessível: Esc, clique no fundo, papel de diálogo.
core/services/tema.ts	Alternância e persistência do tema claro/escuro.
core/services/toast.ts	Avisos flutuantes com fechamento automático.
shared/directives	Revelação ao rolar (IntersectionObserver) e contador numérico animado.
layout/nav, layout/footer	Menu lateral/superior com item ativo; rodapé com contatos e aviso de fã.

10.4 Sistema de design

Cores e tokens

- Azuis #0a7ab8, #006ca6, #004d8b e #013a68 (águas)
- Verdes #58c162, #2f9a46 e #036800 (mata)
- Variáveis CSS de superfície, texto, borda e chip, redefinidas no tema escuro
- 13 arquivos de estilo por responsabilidade (tokens, layout, botões, cartões, modal, formulários, admin, autenticação, feedback, tema, animações e tema escuro)

Tipografia

- Fraunces (serifa) em títulos
- Inter (variável) no texto e na interface
- Fontes empacotadas no build, sem CDN

Componentes

- Cartão com imagem, degradê e título sobreposto
- Bilhete com furos e linha pontilhada
- Modal, botões, abas em cápsula, tabelas, métricas, toasts

Movimento

- Entrada escalonada ao rolar
- Transição entre páginas, esqueletos, contadores
- Tudo desativado com movimento reduzido

10.5 Responsividade

As grades usam `repeat(auto-fill, minmax(min(100%, 290px), 1fr))`; contêineres têm `min-width: 0` e `overflow-wrap: anywhere`; tabelas rolam dentro do próprio quadro; imagens de cartão usam `object-fit: cover`. Assim, nenhum conteúdo ultrapassa os limites do seu cartão. Verificado sem rolagem horizontal em 390 px e 1280 px.

10.6 Tema claro e escuro

O serviço `Tema` grava `data-tema` na raiz do documento e a escolha em `localStorage`. O script externo `tema-init.js` (compatível com a CSP, que proíbe scripts inline) define o tema antes do Angular iniciar, evitando o flash. A troca usa uma transição curta de cores.

CAPÍTULO 11

Back-end e API

11.1 Pipeline de requisição

`helmet` (cabeçalhos e CSP) → `cors` → `express.json` (limite de 50 kB) → limitador de taxa (300/min por IP) → rotas. Nas rotas de autenticação aplica-se um segundo limite (50 por 15 min). Rotas protegidas passam por `auth` (valida o JWT e carrega o usuário) e, no painel, pelo middleware de superadmin. Um tratador de erros final converte exceções em JSON padronizado.

11.2 Contrato da API

Método	Rota	Acesso	Descrição
GET	/api/health	Público	Verificação de saúde
GET	/api/parque	Público	Funcionamento, contatos, localização, redes
GET	/api/atividades	Público	Atividades de lazer
GET	/api/ingressos/tipos	Público	Tipos de ingresso ativos
POST	/api/auth/registro	Público	Cadastro; retorna token e usuário (201)
POST	/api/auth/login	Público	Login (200) ou 401
POST	/api/auth/esqueci-senha	Público	Resposta genérica
GET	/api/auth/me	Usuário	Perfil atual com papel
POST	/api/pedidos	Usuário	Cria pedido e envia e-mail (201)
GET	/api/pedidos	Usuário	Lista os próprios pedidos
GET	/api/pedidos/:id/qrcode	Usuário	Pedido com QR Code (data URL PNG)
POST	/api/pedidos/:id/reenviar	Usuário	Reenvia o comprovante
GET	/api/admin/resumo	Superadmin	Métricas gerais
GET	/api/admin/pedidos	Superadmin	Pedidos de todos (até 500)
GET, PATCH	/api/admin/usuarios[/:id]	Superadmin	Lista; altera papel
GET, POST, PUT	/api/admin/tipos[/:id]	Superadmin	Gestão de tipos de ingresso
POST, PUT, DELETE	/api/admin/atividades[/:id]	Superadmin	Gestão de atividades

11.3 Exemplo

```
POST /api/pedidos
Authorization: Bearer <token>
{ "tipoId": "inteira", "quantidade": 2, "dataVisita": "2026-11-15" }

201 Created
{ "id": 1, "codigo": "CPI-8F3A21C9D0", "tipoNome": "Inteira", "quantidade": 2,
  "dataVisita": "2026-11-15", "totalCentavos": 20000, "emailEnviado": true }
```

11.4 Códigos de resposta e erros

Erros seguem o formato `{ "erro": "mensagem", "campos": { ... } }`.

Código	Uso
400	Validação de entrada, tipo inexistente, data no passado, JSON inválido
401	Token ausente, inválido ou expirado; credenciais incorretas
403	Acesso restrito a superadmin
404	Recurso inexistente ou de outro usuário
409	E-mail já cadastrado
429	Limite de requisições excedido
502	Falha no envio de e-mail durante o reenvio
500	Erro interno (sem detalhes ao cliente)

11.5 Regras de validação (Zod)

Entrada	Regras
Registro	nome ≥ 3; e-mail válido (minúsculo); telefone 10-11 dígitos; senha 8-72; confirmação igual
Login	e-mail válido; senha 1-72
Pedido	tipoId existente e ativo; quantidade inteira 1-10; dataVisita AAAA-MM-DD
Tipo de ingresso	nome 2-60; preço inteiro 0-1.000.000 centavos; descrição 5-600; imagem imagens/arquivo; ativo booleano
Atividade	título 2-60; descrição 10-1500; imagem imagens/arquivo

CAPÍTULO 12

Segurança

A segurança foi tratada em camadas e de forma **orientada a ameaças**: primeiro o modelo STRIDE, depois o mapeamento com o OWASP Top 10 (2021) e, por fim, as recomendações de implantação.

12.1 Modelo de ameaças (STRIDE)

Ameaça	Cenário	Controles	Risco residual
Spoofing (falsificação)	Login de outro usuário	Hash bcrypt, limite de tentativas, JWT assinado com algoritmo fixado, mensagens genéricas	Sem MFA
Tampering (adulteração)	Preço e total do pedido	Total calculado no servidor; SQL parametrizado; validação Zod	Baixo
Repudiation (repúdio)	Negar uma compra ou ação administrativa	Pedidos persistidos com data e dono	Sem trilha de auditoria de ações do admin
Information disclosure (vazamento)	Dados de outros usuários; hashes; versão do servidor	Pedido filtrado por dono; hash nunca retorna; X-Powered-By removido; erros 500 genéricos	Token em localStorage
Denial of service (negação)	Inundar a API ou o login	Limite por IP; corpo de 50 kB; healthcheck e restart	Sem proteção distribuída
Elevation of privilege (elevação)	Usuário virar superadmin	Cadastro sempre cria usuario; papel só por admin ou CLI; middleware em /api/admin; usuário recarregado a cada requisição	Baixo

12.2 Mapeamento OWASP Top 10 (2021)

Categoria	Controle aplicado
A01 Quebra de controle de acesso	Middleware de superadmin; pedidos filtrados por dono; guarda de rota no front como conveniência.
A02 Falhas criptográficas	bcrypt custo 12; JWT HS256 com segredo longo obrigatório; HTTPS recomendado na borda.
A03 Injeção	SQL parametrizado; validação Zod; Angular escapa saídas; e-mail escapa HTML.
A04 Design inseguro	Regras de negócio no servidor; modelo de ameaças STRIDE; endpoints mínimos.

Categoria	Controle aplicado
A05 Configuração incorreta	Helmet, CSP restritiva, nginx com cabeçalhos, contêineres endurecidos, segredos por ambiente.
A06 Componentes vulneráveis	Dependências mínimas e fixadas por lockfile; npm audit sem vulnerabilidades no momento da entrega.
A07 Falhas de identificação e autenticação	Limite de taxa nas rotas de autenticação; política de senha; hash falso contra enumeração.
A08 Falhas de integridade de software e dados	Builds com npm ci (lockfile); imagens oficiais; sem scripts remotos (CSP).
A09 Falhas de registro e monitoramento	Logs de erro no stdout; healthcheck. Evolução: auditoria de ações administrativas.
A10 SSRF	A API não busca URLs informadas pelo usuário.

12.3 Privacidade e LGPD

- **Finalidade e minimização:** coleta somente nome, e-mail, telefone e senha (em hash), apenas para cadastro e compra.
- **Segurança:** senhas nunca são retornadas; comunicação deve ocorrer sobre HTTPS na borda.
- **Natureza do projeto:** os dados são fictícios e o sistema não deve receber dados pessoais reais em produção sem política de privacidade, base legal e canal de atendimento ao titular (Lei 13.709/2018).

12.4 Cabeçalhos aplicados

Cabeçalho	Valor	Objetivo
Content-Security-Policy	default-src 'self'; script-src 'self'; frame-ancestors 'none'; frame-src google.com; object-src 'none'	Bloquear scripts externos e embutir o sistema em outros sites
X-Content-Type-Options	nosniff	Impedir interpretação incorreta de tipos
X-Frame-Options	DENY	Clickjacking
Referrer-Policy	no-referrer	Não vazar URLs
Permissions-Policy	geolocation, camera e microfone desativados	Reduzir permissões do navegador

Pontos de atenção. (1) O token fica em `localStorage`; em produção o ideal é cookie `HttpOnly` com proteção CSRF. (2) HTTPS e HSTS devem ser providos por um proxy reverso na frente do compose. (3) Não há MFA nem trilha de auditoria administrativa.

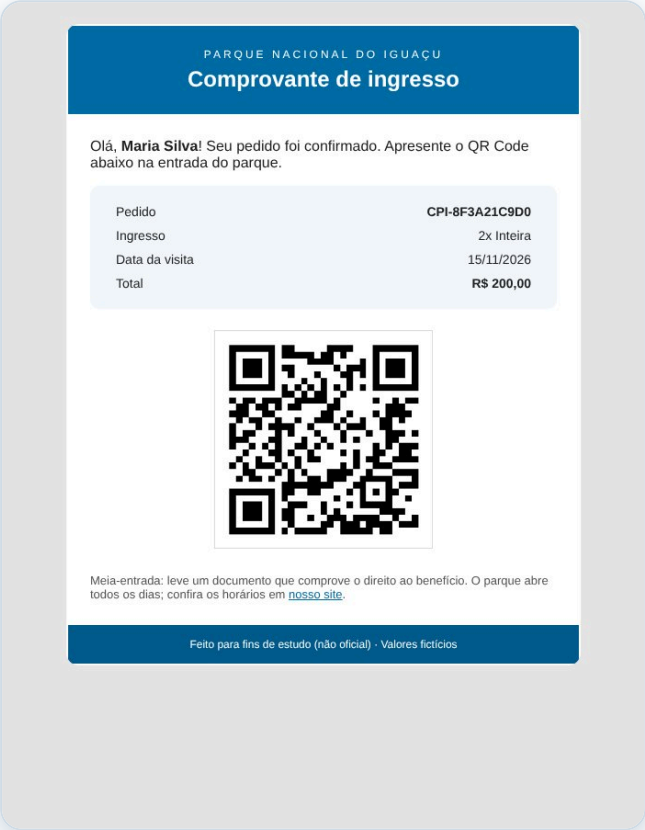
12.5 Recomendações de implantação

- Gerar JWT_SECRET com `openssl rand -hex 32` e mantê-lo fora do Git.
- Publicar atrás de HTTPS (Caddy, Traefik ou nginx) com HSTS.
- Usar SMTP autenticado real, fazer backups do volume dados e restringir o acesso ao terminal do host.

CAPÍTULO 13

E-mail e comprovante

Ao concluir o pedido, a API gera o QR Code (PNG, com código, tipo, quantidade e data) e envia um e-mail HTML via SMTP. O QR Code vai como **anexo inline** (`cid:qrcode`), o que funciona nos principais clientes sem depender de imagens remotas.



E-mail de comprovante de ingresso.

13.1 Conteúdo

- Assunto: *Seu ingresso - Parque Nacional do Iguaçu (CPI-XXXXXXXXXX)*.
- Saudação com o nome do usuário, escapado contra HTML.
- Tabela com código, ingresso, data da visita e total.
- QR Code e orientação sobre meia-entrada.
- Rodapé informando que é projeto de estudo com valores fictícios.

13.2 Configuração

Variável	Padrão no compose	Uso
SMTP_HOST	mailpit	Servidor SMTP (vazio: apenas registra no log)
SMTP_PORT	1025	Porta
SMTP_SECURE	false	TLS implícito (porta 465)
SMTP_USER / SMTP_PASS	vazio	Credenciais
MAIL_FROM	Cataratas do Iguaçu <nao-responda@...>	Remetente
PUBLIC_URL	http://localhost:8080	Link do site no e-mail

13.3 Comportamento em falhas

O transporte tem tempos limites de conexão e leitura. Em caso de erro, o log registra a falha, o pedido continua válido e fica *pendente* (RN16). O usuário pode reenviar pela lista de pedidos ou pela janela do QR Code. Em demonstração, os e-mails ficam retidos no Mailpit (porta 8025) e nenhum e-mail sai para a internet.

CAPÍTULO 14

Administração e CLI

14.1 Painel administrativo

Usuários com papel `superadmin` veem o item *Admin* e acessam `/admin`. A rota é protegida no front pelo `adminGuard` e, de forma definitiva, na API (403 para outros papéis).

Aba	Funções
Resumo	Usuários, pedidos, ingressos vendidos, pedidos do dia e receita fictícia, com contadores animados.
Pedidos	Código, cliente, ingresso, data, total e status do e-mail.
Usuários	Quantidade de pedidos e troca de papel (exceto o próprio).
Ingressos	Criar e editar tipos: nome, preço, descrição, imagem e disponibilidade.
Lazer	Criar, editar e excluir atividades.

14.2 CLI do desenvolvedor

O desenvolvedor gerencia superadmins diretamente no terminal, sem credencial padrão e sem rota pública (ADR-10). A senha, quando omitida, é solicitada de forma oculta, evitando o histórico do shell.

Comando	Efeito
criar	Cria superadmin (ou promove usuário existente). Opções --email, --nome, --senha, --telefone.
atualizar	Altera nome, e-mail (--novo-email), telefone ou senha de um superadmin.
promover	Torna um usuário existente superadmin.
rebaixar	Remove o papel; bloqueia o último superadmin sem --forcar sim.
listar	Lista os superadmins.

```
docker compose exec backend node src/cli.js superadmin criar \
  --email admin@exemplo.com --nome "Admin Geral"
docker compose exec backend node src/cli.js superadmin atualizar \
  --email admin@exemplo.com --novo-email root@exemplo.com
docker compose exec backend node src/cli.js superadmin promover --email maria@exemplo.com
docker compose exec backend node src/cli.js superadmin listar
```

Como a API recarrega o usuário do banco a cada requisição, mudanças de papel feitas pela CLI valem imediatamente.



PARTE IV

Qualidade, operação e encerramento

Como sabemos que funciona e como operar: testes, rastreabilidade, implantação, UX e limitações.

15 · Estratégia e plano de testes

16 · Matriz de rastreabilidade

17 · Implantação e operação

18 · Experiência do usuário e acessibilidade

19 · Riscos residuais, limitações e evolução

CAPÍTULO 15

Estratégia e plano de testes

15.1 Estratégia

Nível	Abordagem	Ferramenta	Situação
API (integração)	Servidor real em porta efêmera, banco em memória, e-mail simulado	node --test (nativo)	Automatizado, no repositório
CLI	Execução da função run() contra banco em memória	node --test	Automatizado, no repositório
Unidade (utilitários)	Slug, datas, senhas, ambiente, validação e argumentos da CLI	node --test	Automatizado, no repositório
Análise estática	ESLint (back-end, front-end com regras de acessibilidade de template, E2E)	ESLint, angular-eslint	Automatizado; executado no CI
Interface ponta a ponta	Cadastro, login com retorno, compra, QR, painel, tema e responsividade, em desktop (1280 px) e celular (390 px)	Playwright (Chromium), pasta e2e	Automatizado, no repositório
Integração contínua	Testes da API, build do Angular, ponta a ponta, npm audit, docker compose config e build	GitHub Actions (.github/workflows/ci.yml)	Definido; executa a cada push, ainda não executado no GitHub
Infraestrutura	Build das imagens e subida do compose	Docker (no CI)	Não validado no ambiente de entrega (sem acesso ao registro npm); coberto pelo CI

15.2 Casos de teste automatizados

ID	Caso	Resultado esperado	RF / RN
TC01	Conteúdo público	Health ok; 6 atividades; 2 tipos ativos; contatos do parque	RF01-RF04
TC02	Cadastro com confirmação diferente	400	RF07; RN02
TC03	Cadastro válido com e-mail em maiúsculas	201; e-mail salvo em minúsculas	RF07; RN01

ID	Caso	Resultado esperado	RF / RN
TC04	Cadastro com e-mail repetido	409	RF07; RN01
TC05	Login com senha errada / correta	401 / 200 com token; /auth/me retorna o perfil	RF08
TC06	Acesso sem token	401	RF08
TC07	Pedido sem token	401	RF12
TC08	Pedido com tipo inexistente ou data passada	400	RF12; RN05, RN08
TC09	Pedido válido (3 × meia)	201; total 15.000 centavos; e-mail enviado	RF12-RF14; RN06
TC10	Reenvio e QR Code	200; segundo e-mail; QR em data URL PNG; pedido alheio 404	RF13, RF15; RN11
TC11	Painel com usuário comum / sem token	403 / 401	RF17-RF21
TC12	Painel com superadmin	Resumo e listas corretos	RF17-RF19
TC13	Criar tipo; entrada inválida	201 com slug; 400	RF20
TC14	Desativar tipo	Some da vitrine pública	RF20; RN08
TC15	Criar e excluir atividade	Lista pública reflete (7, depois 6)	RF21
TC16	CLI: criar, atualizar, promover, rebaixar, listar	Papéis e dados conforme comandos	RF22; RN12
TC17	CLI: senha curta; último superadmin; usuário inexistente	Erros específicos	RF22; RN02, RN14
TC18	Unidade: slugify e uniqueSlug	Sem acentos, nunca vazio, contador em colisão	RF20, RF21
TC19	Unidade: senhas e usuário inexistente	Hash confere; inexistente nunca autentica	RN02; RNF09
TC20	Unidade: ambiente em produção	Exige JWT_SECRET de 32+ caracteres	RNF15
TC21	Unidade: validate e parseArguments	HttpError 400 com campos; opção sem valor falha	RN01-RN03

15.3 Cenários ponta a ponta (Playwright)

ID	Cenário	Verificações	RF
ES01	Páginas públicas	Seis rotas carregam sem erro de console e sem rolagem horizontal	RF01-RF05; OB2
ES02	Lazer	Cartão abre os detalhes e fecha com Esc	RF03
ES03	Tema	Alterna, persiste após recarregar e segue prefers-color-scheme	RF06
ES04	Cadastro e compra	Compra sem login leva ao login com retorno; cadastro, retorno a Ingressos, compra, QR Code e pedido listado	RF07, RF11-RF16

ID	Cenário	Verificações	RF
ES05	Validação de formulário	Senhas diferentes bloqueiam o cadastro	RF07; RN02
ES06	Painel administrativo	Usuário comum é barrado; superadmin criado pela CLI acessa, percorre as abas e cria uma atividade	RF17-RF22

Cada cenário roda em dois perfis (desktop e celular), totalizando 12 execuções.

15.4 Resultado

```
cd backend && npm test
tests 12 · pass 12 · fail 0

cd backend && npm run lint
cd frontend && npm run lint

cd e2e && npx playwright test
12 passed (6 cenários × 2 perfis)
```

15.5 Defeitos encontrados pelos testes

A verificação em navegador real revelou e corrigiu defeitos que a leitura do código não mostrava: o hero sobrepondo o menu, ícones do botão de tema deslocados por CSS legado, campos do login acinzentados no tema escuro e rótulos acessíveis (`aria-label`) que substituíam o texto visível de links e campos, contrariando o critério 2.5.3 da WCAG (rótulo no nome). A análise estática encontrou ainda asserções não nulas no redirecionamento seguro e um rótulo sem controle associado no seletor de quantidade, ambos corrigidos.

15.6 Lacunas e plano

- Executar o fluxo de CI no GitHub e acompanhar o resultado a cada alteração.
- Testes de carga e de segurança (SAST); a varredura de dependências (npm audit) já faz parte do CI.
- Auditoria de acessibilidade (WCAG 2.2) com ferramentas automatizadas e leitor de tela.

CAPÍTULO 16

Matriz de rastreabilidade

A matriz liga cada requisito a seus casos de uso, regras de negócio, pontos de implementação e casos de teste, permitindo analisar o impacto de mudanças e comprovar cobertura.

Requisito	Caso de uso	Regras	Implementação	Verificação
RF01-RF04	UC01	RN08, RN15	GET /api/parque, /atividades, /ingressos/tipos	TC01, TC14, ES01
RF05	UC01	RN17, RN18	estático (rodapé, Sobre, Ingressos)	ES01
RF06	UC02	RN19	local (localStorage)	ES03
RF07	UC03	RN01-RN03, RN12	POST /api/auth/registro	TC02-TC04, ES04, ES05
RF08-RF09	UC04	—	POST /api/auth/login, GET /me	TC05, TC06
RF10	UC04	—	POST /api/auth/esqueci-senha	Inspeção (sem teste automatizado)
RF11	UC05	—	local (returnUrl seguro)	ES04
RF12	UC05	RN04-RN08, RN10	POST /api/pedidos	TC07-TC09, ES04
RF13-RF15	UC05, UC06	RN10, RN11, RN16	GET /pedidos/:id/qrcode, POST /reenviar	TC09, TC10, ES04
RF16	UC07	RN11	GET /api/pedidos	TC09, TC10, ES04
RF17-RF19	UC08	RN13	GET /admin/resumo, /pedidos, /usuarios; PATCH /usuarios/:id	TC11, TC12, ES06
RF20	UC09	RN08	GET, POST, PUT /api/admin/tipos	TC13, TC14, ES06
RF21	UC10	—	POST, PUT, DELETE /api/admin/atividades	TC15, ES06
RF22	UC11	RN12, RN14	CLI	TC16, TC17, ES06

Cobertura: 22 de 22 requisitos funcionais possuem caso de uso; 21 têm verificação automatizada (API, CLI ou ponta a ponta). A exceção é o RF10 (recuperação de senha simulada), verificado apenas por inspeção.

CAPÍTULO 17

Implantação e operação

17.1 Serviços

Serviço	Imagem	Porta	Observações
frontend	Build em Node 22 Alpine; execução em nginx-unprivileged	8080	Entrega o Angular e faz proxy de /api; aguarda o back-end saudável.
backend	Node 22 Alpine	3000 (interna)	Usuário node, somente leitura, volume dados, healthcheck.
mailpit	axllent/mailpit	8025	Caixa de entrada de demonstração.

17.2 Execução

```
cp .env.example .env
docker compose up --build
docker compose exec backend node src/cli.js superadmin criar --email voce@exemplo.com --nome
"Seu Nome"
```

Antes de subir, edite o `.env` e defina `JWT_SECRET` (por exemplo, com `openssl rand -hex 32`). Site em `http://localhost:8080` e e-mails de demonstração em `http://localhost:8025`.

17.3 Variáveis de ambiente

Variável	Obrigatória	Descrição
JWT_SECRET	Sim (produção)	Segredo de assinatura; 32 caracteres ou mais
PORT	Não	Porta publicada do site (8080)
MAILPIT_PORT	Não	Porta da UI do Mailpit (8025)
PUBLIC_URL	Não	URL do site usada no e-mail
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS	Não	SMTP real (padrão: Mailpit)
MAIL_FROM	Não	Remetente

Variável	Obrigatória	Descrição
DATA_DIR	Não	Pasta do banco (padrão /data no contêiner)

17.4 Operação

Tarefa	Comando
Logs da API	docker compose logs -f backend
Parar mantendo dados	docker compose down
Atualizar	docker compose up --build -d
Apagar dados (destrutivo)	docker compose down -v
Backup do banco	docker compose stop backend; docker compose cp backend:/data/cataratas.db ./backup.db; docker compose start backend
Verificar saúde	curl http://localhost:8080/api/health

17.5 Diagnóstico

Sintoma	Causa provável	Ação
Compose recusa subir: defina JWT_SECRET	Variável ausente	Criar .env a partir de .env.example
API reinicia em loop	JWT_SECRET curto (< 32) em produção	Gerar segredo com openssl rand -hex 32
Comprovante pendente	SMTP indisponível ou mal configurado	Verificar SMTP_*; usar Reenviar; consultar os logs
Nada na caixa do Mailpit	SMTP_HOST aponta para outro servidor	Usar SMTP_HOST=mailpit na demonstração
Erro 429	Limite de requisições	Aguardar a janela e reduzir a frequência
Painel inacessível	Usuário não é superadmin	Promover pela CLI (promover --email ...)

Validação de infraestrutura. Os Dockerfiles e o compose foram escritos e revisados, mas o build completo não pôde ser concluído no ambiente de desenvolvimento deste projeto, porque o contêiner de build não tinha acesso ao registro npm. Antes de publicar, execute `docker compose up --build` em uma máquina com internet.

CAPÍTULO 18

Experiência do usuário e acessibilidade

18.1 Heurísticas de Nielsen

Heurística	Como é atendida
Visibilidade do status do sistema	Esqueletos de carregamento, botões com estado (Processando...), status do e-mail, toasts de confirmação.
Correspondência com o mundo real	Linguagem em português, bilhetes com furos e linha pontilhada, vocabulário do parque.
Controle e liberdade	Fechar janelas por botão, fundo ou Esc; cancelar compra; alternar tema.
Consistência e padrões	Sistema de design único, mesmos botões, cartões e modais em todas as telas.
Prevenção de erros	Quantidade limitada de 1 a 10; data mínima; senha com mostrar/ocultar; confirmação ao excluir atividade.
Reconhecimento em vez de memorização	Ícones junto ao texto, item de menu ativo, breadcrumbs implícitos pelo título.
Flexibilidade e eficiência	Retorno automático à página de origem após login; reenvio rápido do comprovante.
Design estético e minimalista	Hierarquia tipográfica, espaço em branco, imagem como protagonista.
Ajuda a reconhecer e corrigir erros	Mensagens específicas junto aos campos e avisos claros em falhas de e-mail.
Ajuda e documentação	Textos de apoio nas páginas e este documento.

18.2 Acessibilidade (WCAG 2.2)

Prática	Aplicação	Situação
Texto alternativo	Imagens de conteúdo com alt descritivo	Aplicado
Rótulos de formulário	label associado a cada campo; aria-label complementar	Aplicado
Navegação por teclado e foco	Foco visível; Esc fecha janelas; botões nativos	Aplicado

Prática	Aplicação	Situação
Semântica e ARIA	role="dialog", aria-modal, aria-live em avisos, aria-pressed no tema	Aplicado
Movimento reduzido	prefers-reduced-motion desativa animações	Aplicado
Tamanho de alvos e reflow	Botões de 38 a 44 px; layout fluido sem rolagem horizontal	Aplicado
Nome acessível contém o texto visível (2.5.3)	aria-label redundantes removidos; links e campos usam o texto visível ou label	Aplicado
Operação por teclado	Botão de menu e "Esqueceu sua senha?" focáveis e acionáveis por Enter	Aplicado
Rótulos associados a controles (1.3.1, 4.1.2)	Grupo de quantidade rotulado por aria-labelledby; regras de acessibilidade de template no ESLint	Aplicado
Contraste de cores	Paleta com texto claro sobre azul escuro e escuro sobre claro	Verificação visual; medição formal pendente
Auditoria com leitor de tela	—	Pendente

> Este documento não declara conformidade formal com a WCAG. As práticas acima foram aplicadas e inspecionadas, e a auditoria completa consta como evolução.

CAPÍTULO 19

Riscos residuais, limitações e evolução

19.1 Limitações conhecidas

ID	Limitação	Impacto	Tratamento sugerido
L1	Pagamento fictício	Sem receita real	Integrar gateway e conciliar pedidos
L2	Recuperação de senha simulada	Usuário não consegue redefinir a senha sozinho	Tokens de uso único por e-mail
L3	QR Code sem validação na portaria	Sem controle de uso do ingresso	App/portal de leitura e estado "utilizado"
L4	Sem controle de capacidade por dia	Venda ilimitada	Estoque por data
L5	Token em localStorage	Exposição em caso de XSS	Cookie HttpOnly + CSRF
L6	SQLite com uma instância	Escala limitada	PostgreSQL e migrações versionadas
L7	Sem trilha de auditoria administrativa	Pouca rastreabilidade de ações do admin	Registro de eventos administrativos
L8	Build do Docker não validado no sandbox	Risco de ajuste fino na primeira execução	CI já define o build; executar em máquina com internet

19.2 Roteiro de evolução

Produto

- Redefinição de senha por e-mail
- Leitor de QR Code e baixa do ingresso
- Calendário e capacidade por data
- Internacionalização (inglês e espanhol)

Engenharia

- Integração contínua com testes ponta a ponta
- PostgreSQL e migrações
- Cookie HttpOnly, CSRF e HSTS
- Observabilidade e logs estruturados

19.3 Lições do projeto

- Documentar requisitos com critérios verificáveis tornou os testes diretos.
- Medir e verificar em navegador real antes de afirmar: o build apontou imagens de 15 MB (agora centenas de kB) e a interface revelou defeitos invisíveis na leitura do código.

- Refatorar com rede de segurança: testes, E2E e comparação visual pixel a pixel permitiram reorganizar todo o código sem alterar o comportamento.
- Automatizar o estilo: regras de lint impedem a volta de comentários, duplicações e problemas de acessibilidade.

APÊNDICE A

Glossário

Termo	Significado
ADR	Registro de decisão arquitetural.
API REST	Interface de comunicação sobre HTTP com recursos e verbos.
bcrypt	Algoritmo de hash de senhas com custo configurável.
C4	Modelo de diagramas de arquitetura: contexto, contêineres, componentes e código.
CLI	Interface de linha de comando.
CSP	Content Security Policy: política que limita as origens de conteúdo.
Docker Compose	Descrição e execução de múltiplos contêineres.
JWT	JSON Web Token: credencial assinada usada para sessões.
LGPD	Lei Geral de Proteção de Dados (Lei 13.709/2018).
Mailpit	Servidor SMTP de testes com caixa de entrada web.
MoSCoW	Técnica de priorização: Must, Should, Could, Won't.
OWASP	Comunidade que mantém guias de segurança de aplicações.
Papel	Perfil de acesso: usuário ou superadmin.
QR Code	Código bidimensional lido por câmeras.
RF / RNF / RN	Requisito funcional, não funcional e regra de negócio.
SAD / SDD / SRS	Descrição de arquitetura, de projeto e especificação de requisitos de software.
SMTP	Protocolo de envio de e-mails.
SPA	Aplicação de página única.
STRIDE	Modelo de ameaças: Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege.
Superadmin	Usuário com acesso ao painel administrativo.
WCAG	Diretrizes de acessibilidade para conteúdo web.

APÊNDICE B

Referências

Referência	Uso neste documento
ISO/IEC/IEEE 29148:2018 · Systems and software engineering, Life cycle processes, Requirements engineering	Estrutura e qualidade dos requisitos (SRS)
ISO/IEC/IEEE 42010:2022 · Architecture description	Visões de arquitetura e preocupações das partes interessadas
ISO/IEC 25010 · Systems and software Quality Requirements and Evaluation (SQuaRE), quality model	Classificação dos requisitos não funcionais
Simon Brown · The C4 model for visualising software architecture	Diagramas de contexto, contêineres e componentes
Michael Nygard · Documenting Architecture Decisions	Formato dos ADRs
OWASP Top 10 (2021) e OWASP ASVS	Mapeamento de controles de segurança
Microsoft STRIDE · Threat modeling	Modelo de ameaças
W3C · Web Content Accessibility Guidelines (WCAG) 2.2	Práticas de acessibilidade
Jakob Nielsen · 10 Usability Heuristics for User Interface Design	Avaliação de UX
RFC 7519 · JSON Web Token (JWT); RFC 5321 · SMTP	Sessões e e-mail
Lei nº 13.709/2018 (LGPD)	Privacidade e proteção de dados
Documentação oficial: Angular, Node.js, Express, Docker, nginx, SQLite	Tecnologias do projeto

As normas são citadas como orientação de boas práticas. O documento não alega certificação ou conformidade formal com nenhuma delas.



Obrigado por visitar

Este projeto nasceu do carinho por um dos lugares mais impressionantes do Brasil. Que as quedas do Iguaçu continuem inspirando.

Projeto de fã e portfólio. Não representa o site real das Cataratas do Iguaçu.